

VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



URBANPULSE-STREAMING PLATFORM FOR LIVE CITY
ACTIVITY AND ANOMALY DETECTION

By
Chau Thinh - ITCSIU22240

*A thesis submitted to the School of Computer Science & Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Computer Science*

Ho Chi Minh City, Vietnam
June 2026

URBANPULSE-STREAMING PLATFORM FOR LIVE CITY ACTIVITY AND ANOMALY DETECTION

APPROVED BY:

MSc. Dang Tam Nhan

Acknowledgments

I would like to express my deepest gratitude to the Vietmap Solutions Team for their generous financial sponsorship and invaluable technical assistance throughout the course of this thesis. Their support provided essential access to high-frequency traffic data and API quotas, which were critical to the development, testing, and evaluation of the UrbanPulse platform. Without their trust and contribution, realizing the real-time detection capabilities of this system would not have been possible.

Furthermore, I am profoundly grateful for the professional guidance and domain expertise shared by MSc. Dang Tam Nhan. I also wish to extend my sincere thanks to my friends for their unwavering encouragement and proactive support.

This collaborative journey has significantly enriched my understanding of Intelligent Transportation Systems (ITS) and has served as a cornerstone of this project.

Table of Contents

List of Tables	vii
List of Figures	x
List of Algorithms	xi
List of Listings	xii
Abstract	xiii
1 INTRODUCTION	1
1.1 Problem Statement	1
1.2 Scope	2
1.3 Objectives	2
2 RELATED WORK	4
2.1 Current Advancements	4
2.2 Research Gap	4
2.3 Theoretical Background	5
2.3.1 The Medallion Lakehouse Architecture	5
2.3.2 Anomaly Detection Logic	6
3 METHODOLOGY	8
3.1 Requirement Analysis	8
3.1.1 Functional Requirements	8
3.1.2 Non-Functional Requirements	9
3.1.3 Use Case Description	9
3.1.4 Use Case Diagram	9
3.1.5 System Overview	10
3.2 Data Acquisition and Ingestion Strategy	12
3.2.1 Traffic Telemetry — VietMap Directions API	12
3.2.2 Meteorological Context — Open-Meteo API	12
3.3 Stream Processing and Medallion Architecture	13
3.3.1 Bronze Layer — Durable Event Archival	13
3.3.2 Silver Layer — Incremental Micro-Batch Processing	14
3.3.3 Gold Layer — Hourly Aggregation	15
3.3.4 Online Feature Store — Real-Time Postgres Path	15
3.4 Dual-Signal Anomaly Detection Framework	16
3.4.1 Signal 1 — Z-Score Online Detector	16
3.4.2 Signal 2 — Isolation Forest Batch Detector	17
3.4.3 Dual-Signal Fusion	18
3.5 Knowledge-Enriched Inference via RAG	20
3.5.1 Role in the System (Why)	20
3.5.2 Implementation Pipeline (Input – Configuration – Output)	20
3.5.3 Feedback Loop and Fine-Tuning Data Collection	21
3.6 Evaluation Framework	21

3.6.1	Temporal Integrity	22
3.6.2	Reproducibility	22
3.6.3	Performance Metrics	22
4	PROTOTYPING	23
4.1	Technology Stack Selection	23
4.1.1	Message Broker: Redpanda over Apache Kafka	23
4.1.2	Storage: Iceberg + Nessie + MinIO	24
4.1.3	LLM: Ollama with Qwen2.5:3b	24
4.2	Service Topology Design	24
4.3	Data Schema Design	25
4.3.1	Kafka Message Schema	25
4.3.2	Iceberg Table Schemas	26
4.3.3	Weather Schema	26
4.3.4	PostgreSQL Online Store	26
4.4	End-to-End Data Flow Through the Medallion Layers	26
4.4.1	Data Versioning with Nessie and Iceberg	27
4.5	Infrastructure Prototyping	27
5	IMPLEMENTATION	28
5.1	Ingestion Service	28
5.1.1	Traffic Ingestion Service	28
5.1.2	Weather Ingestion Service	29
5.2	Streaming Service	30
5.2.1	Traffic Processor	31
5.2.2	Weather Processor	31
5.3	Online Service	32
5.3.1	Message Ingestion	32
5.3.2	Welford Accumulator	32
5.3.3	Z-Score Anomaly Signal	33
5.3.4	Crash Recovery	33
5.4	Batch Pipeline	33
5.4.1	Bronze-to-Silver Promotion	33
5.4.2	Silver-to-Gold Aggregation	34
5.4.3	Baseline Learning	34
5.4.4	Anomaly Alerter	34
5.5	ML Service	35
5.5.1	Feature Engineering	35
5.5.2	IsolationForest Configuration	35
5.6	Serving Layer	35
5.6.1	Endpoint Overview	35
5.6.2	SSE Streaming Architecture	35
5.6.3	RAG-Enriched Inference	36
5.6.4	Dual-Signal Fusion in Serving	39
5.6.5	Frontend Dashboard	39

6	RESULT	42
6.1	Experimental Setup	42
6.1.1	Dataset	42
6.1.2	Evaluation Approach	42
6.2	Dual-Signal Detection Results	43
6.2.1	Per-Route Metrics	43
6.2.2	Aggregate Findings	45
6.2.3	Production Anomaly Distribution	46
6.2.4	Interpretation	46
6.3	System Performance	47
6.3.1	Grafana Observability Dashboards	47
6.3.2	End-to-End Ingestion Latency	49
6.3.3	Batch Pipeline Throughput	49
6.3.4	SSE Delivery Latency	50
6.3.5	LLM Inference Latency	50
6.3.6	Summary of Performance Against Requirements	50
7	DISCUSSION	52
7.1	Analysis of Results	52
7.2	Strengths	53
7.3	Limitations	53
7.4	Future Work	54
8	CONCLUSION	55
A	SUPPLEMENTARY ARTIFACTS	60
A.1	Route and Zone Configuration	60
A.2	Runtime Data Artifacts	60
A.3	Serving API Surface	61
A.4	Reproducibility Commands	61
A.5	Evaluation Artifacts	61

List of Tables

2.1	Comparison of data processing architectural patterns by data-flow characteristics (✓ yes, ✗ no).	5
2.2	Comparison of analytical storage paradigms by data management capabilities (✓ yes, ✗ no).	6
3.1	Primary use cases and actors in UrbanPulse	9
3.2	Zone-specific Open-Meteo query coordinates.	13
3.3	IsolationForest hyperparameters selected by Optuna TPE (50 trials × 20 routes; median aggregation).	18
3.4	Performance metrics: temporal integrity, reproducibility, and system performance	22
4.1	UrbanPulse technology stack and selection rationale.	24
4.2	Medallion data flow: transformation, ownership, and latency per layer.	26
5.1	Serving API endpoint summary.	36
6.1	Dataset summary for the MLflow training run	43
6.2	Per-route anomaly detection metrics	44
6.3	Aggregate dual-signal metrics across 20 routes.	45
6.4	Measured performance vs. non-functional requirements.	51
A.1	UrbanPulse zone anchors used for route generation.	60
A.2	Primary persisted artifacts for reproducible anomaly decisions.	61
A.3	Serving API endpoints used by the evaluation workflow.	61

List of Figures

2.1	Conceptual overview of the Medallion Lakehouse architecture in UrbanPulse.	5
3.1	Use case diagram for the UrbanPulse platform.	10
3.2	High-level system architecture of UrbanPulse.	11
3.3	Dual-consumer data flow from Redpanda to Lakehouse and Online Feature Store.	14
3.4	Per-route Z-score over a 24-hour observation window (Grafana). Each line represents one directed inter-zone route. Spikes above the dashed threshold θ_r trigger the Z-score anomaly signal.	17
3.5	Optuna hyperparameter tuning validation across 20 routes. <i>Left</i> : tuned vs. default anomaly score gap per route (all points above the diagonal indicate improvement). <i>Right</i> : distribution of score gap delta (tuned – default); all 20 routes show strictly positive improvement (mean +0.023, max +0.047).	19
3.6	MLflow experiment view showing per-route IsolationForest training runs for the traffic-anomaly-detection experiment. Each row corresponds to one directed inter-zone route model.	19
4.1	UrbanPulse complete system architecture: data ingestion from VietMap and Open-Meteo via Redpanda, medallion lakehouse (Bronze/Silver/Gold) on MinIO with Iceberg and Nessie, Online Service writing to PostgreSQL, Batch and ML Services tracked by MLflow, RAG pipeline via ChromaDB and Ollama, and Next.js frontend served through Traefik. .	23
4.2	UrbanPulse service topology	25
5.1	Redpanda Console: vietmap-raw topic confirming continuous ingestion across the observation period.	30
5.2	Prefect flow schedules visualised as a Gantt-style timeline. Each row is one flow; bar width is proportional to schedule interval. Microbatch and Alerter fire every 5 minutes; Gold aggregation runs hourly; ML Retrain with RAG indexing runs every 6 hours.	33
5.3	Prefect 3.x Deployments view: seven active flows covering ingestion, batch promotion, model retraining, RAG indexing, and Telegram alerting. .	34
5.4	AI Anomaly Explanation modal for the Zone 3 → Zone 4 route (Vietnamese output): Qwen2.5:3b structures its response into three sections — <i>Quan sát</i> (heavy_ratio = 4.1%, zscore = −1.53, free-flowing), <i>Nguyên nhân</i> (Friday 18:00 off-peak, industrial/port traffic decrease, stable weather at 26.5°C), and <i>Đánh giá</i> (low-severity, positive free-flow event).	37
5.5	Urban Pulse AI Chat demonstrating bilingual capability: an English query “how’s the traffic now” receives an English response reporting the Southern Port → Western Periurban route at 10.9% heavy ratio and overcast conditions at 29.5°C, retrieved from the PostgreSQL feature store and Open-Meteo feed via Qwen2.5:3b.	38

5.6	Traffic Metrics heatmap: tooltip showing a Z-score-only anomaly on the Coastal → Urban Core route at 08:00 ($Z = 4.22$), Apr 12–15, 2026. . . .	39
5.7	Traffic Metrics heatmap: tooltip showing a dual-signal anomaly (BOTH) on the Coastal → Western route at 15:00 ($Z = 2.02$), Apr 12–15, 2026. . . .	40
5.8	AI Heatmap Analysis panel (Vietnamese — <i>Phân tích Heatmap bằng AI</i>): Qwen2.5:3b identifies two dual-signal anomalies on Zone 3 → Zone 4 at 16:00 and 18:00, attributes them to logistics and industrial traffic, and lists Zone 1 → Zone 3/4 and Zone 2 → Zone 3/4 as single-signal developing congestion candidates.	40
5.9	AI Heatmap Analysis panel (English): the same four-part structure rendered in English confirms Zone 3 → Zone 4 dual-signal anomalies at 16:00 ($Z=3.4$) and 18:00 ($Z=2.5$), with root causes linked to the southern port industrial corridor and evening-shift logistics. Single-signal routes (Zone 1 → Zone 2, Zone 3 → Zone 5) are noted as isolated but significant events.	41
6.1	Dremio EDA query on <code>nessie.gold.traffic_hourly</code> : 20 routes, 76 days covered, confirming the Iceberg Gold table is correctly populated by the batch pipeline.	42
6.2	MLflow experiment <code>traffic-anomaly-detection</code> : 20 per-route runs all completed with status FINISHED in ~ 3.5 s each, confirming the ML Service retraining pipeline is stable.	44
6.3	MLflow per-route metric charts for <code>zone6</code> → <code>zone5</code> : agreement = 0.96, Jaccard = 0.16, IForest rate = 0.01 (tuned contamination), 12 joint anomaly hours from 1,517 samples.	45
6.4	Model agreement (blue) and Jaccard similarity (orange) for five representative routes. Agreement consistently exceeds 0.90; Jaccard variation reflects route-specific traffic pattern stability.	46
6.5	Production Anomalies Per Hour (Apr 12–15, 2026): yellow = Z-Score only, green = IForest only, red = both detectors agree. Morning rush-hour clusters are visible, alongside a late-evening IForest-only spike near 23:00 on Apr 15.	47
6.6	Grafana Online Features dashboard (Apr 13–15, 2026): p50 = 153 ms, p95 = 318 ms, p99 = 336 ms ingestion-to-feature latency. Anomalous routes over time shown in the bottom-right panel.	48
6.7	Grafana Anomaly Detection dashboard (Apr 13–15, 2026): anomalous route count over time (top-left), maximum heavy-ratio Z-Score intensity reaching roughly 9 (top-right), and per-route Z-Score time series for all 20 routes (bottom-right).	48
6.8	Grafana Ingestion Service dashboard: periodic 5-minute burst pattern at 0.08–0.12 req/s confirming correct API polling cadence. Zero errors and zero warnings over the observed window.	49
6.9	Breakdown of <code>ingest_lag_ms</code> : the timestamp <code>ingest_ts</code> is recorded by the Ingestion Service before the VietMap API call and embedded in the Kafka header; the Online Consumer stamps t_{now} after the PostgreSQL UPSERT completes. All five stages run on the same host, keeping network overhead near zero.	49

6.10 Grafana Urban Traffic Analytics dashboard: Bronze→Silver 4,850 rows, Silver→Gold 120 rows, Baseline 3,360 records. Batch job completion logs confirm error-free pipeline execution.	50
--	----

List of Algorithms

1	$(\hat{\mu}, \hat{\sigma}, n) \leftarrow \text{WelfordUpdate}(x, \hat{\mu}_{\text{prev}}, M_{2,\text{prev}}, n_{\text{prev}})$	15
2	$\text{tier}_r \leftarrow \text{SignalFusion}(z_r, f_r)$	20

List of Listings

5.1	Raw API fetch in <code>sources/vietmap.py</code>	29
5.2	Kafka publish in <code>publishers/kafka.py</code>	29
A.1	Excerpt from the route configuration file.	60
A.2	Local reproducibility commands.	61

Abstract

The rapid acceleration of urbanization has necessitated the development of sophisticated municipal monitoring systems capable of processing high-velocity data for real-time decision-making. Conventional urban management frameworks often rely on periodic batch processing, leading to significant temporal gaps and fragmented situational awareness. This thesis presents UrbanPulse, a streaming-first urban intelligence platform designed to ingest, process, and analyze heterogeneous city-scale data streams to provide near-real-time insights into metropolitan dynamics, with a case study deployment scoped to Ho Chi Minh City, Vietnam.

The proposed architecture leverages a distributed event-driven approach using Redpanda, a Kafka-compatible message broker, for low-latency event queuing. A centralized Data Lakehouse—implemented via MinIO, Apache Iceberg, and Project Nessie—is utilized to ensure transactional integrity, schema evolution, and time-travel capabilities for historical analysis. The platform incorporates a specialized ELT pipeline that transforms raw telemetry from VietMap and Open-Meteo APIs into refined inter-zone mobility metrics across Bronze, Silver, and Gold layers.

To bridge the gap between raw data and actionable intelligence, UrbanPulse integrates a dual-signal anomaly detection module. This module combines Welford’s online algorithm for memory-efficient ($O(1)$) dynamic Z-score baselining with an Isolation Forest model to identify complex, non-recurrent traffic incidents. Furthermore, a Retrieval-Augmented Generation (RAG) framework enriches Large Language Model (LLM) outputs with historical anomaly patterns and current meteorological context retrieved from a vector store (ChromaDB). This approach enables automated root-cause analysis without the computational overhead of frequent model retraining.

System performance was validated through rigorous testing on a dataset of 32,302 real-world observations. Results indicate that the streaming-first design achieves a 93.8% dual-signal consensus rate with a median ($p50$) end-to-end latency of just 153ms. This significantly reduces the latency of situational awareness compared to traditional batch-based architectures, providing a scalable, evidence-based foundation for urban operational resilience.

Keywords: Data Engineering, Streaming Systems, Redpanda, Urban Intelligence, Anomaly Detection, Retrieval-Augmented Generation, Ho Chi Minh City.

Chapter 1

INTRODUCTION

1.1 Problem Statement

Urban monitoring systems in modern smart cities are increasingly adopting data-driven architectures.

In the context of the UrbanPulse system, traffic data is ingested from the Vi-etMap API—capturing travel duration and congestion ratios—and processed through a hybrid Medallion Lakehouse architecture combining streaming inference paths and micro-batch analytical layers. Despite this structured approach, interpreting anomalies in a streaming environment introduces several challenges:

- **Signal Inconsistency:** The system utilizes a dual-signal detection framework that combines statistical methods (Z-score) with machine learning models (Isolation Forest). Statistical detection operates on real-time rolling windows, whereas machine learning models rely on periodically refreshed historical features maintained through offline feature pipelines, potentially causing inconsistency in anomaly interpretation.
- **Cold-start and Data Sparsity:** Effective Z-score computation requires a stable historical baseline of road segment activity. During the initial system bootstrap or in low-variance traffic conditions, the lack of robust baseline statistics results in missing or unreliable anomaly signals.
- **Model Staleness:** Machine learning models stored in the MinIO-based model registry are typically retrained on a schedule. This introduces a "staleness" factor, where the model may fail to adapt to sudden distribution shifts caused by unpredicted road incidents or temporary construction.
- **High-Dimensional Latency Constraints:** Ensuring low-latency propagation from the Redpanda broker to the end-user dashboard via Server-Sent Events (SSE) requires a highly optimized serving layer that must reconcile high-frequency ingestion without introducing significant lag.

In practice, both the online and offline processing paths consume the same event stream published to the Redpanda broker, which acts as the system's Single Source of Truth. The online path computes streaming baselines using Welford's online algorithm (memory-efficient, $O(1)$) to produce immediate Z-score signals, while the offline path materialises the same events into the Lakehouse (MinIO + Apache Iceberg) for batch feature engineering and periodic model retraining. This architecture preserves stream-to-batch consistency while allowing ML models to be trained on the authoritative historical dataset.

Therefore, the core problem addressed in this thesis is:

How to design a hybrid low-latency traffic anomaly detection architecture that ensures consistency between real-time signals and historical baselines while maintaining scalability through a unified lakehouse strategy?

1.2 Scope

This thesis focuses on the design and implementation of the "UrbanPulse" platform, an integrated system for real-time traffic monitoring and event detection within Ho Chi Minh City. The scope is defined as follows:

1. Data Ingestion and Sources

- Real-time traffic flow data is sourced from VietMap APIs (20 routes) and supplemented by weather data from the Open-Meteo API for environmental context.
- All observations are published to the Redpanda broker, which serves as the system's Single Source of Truth; both the online Welford-based stream processors and offline lakehouse pipelines consume from this stream to maintain consistency.

2. Technical and Architectural Framework

- The system leverages a Kafka messaging backbone for high-throughput event propagation and fault-tolerant ingestion.
- Data storage and lifecycle management are governed by a Lakehouse pattern utilizing Apache Iceberg for table formats, Project Nessie for catalog versioning, and MinIO as the underlying object store.
- The processing pipeline is implemented in Python, combining streaming computations for online anomaly scoring and batch processing pipelines for data quality validation, feature engineering, model retraining, and the Medallion data governance layers (Bronze $\rightarrow$ Silver $\rightarrow$ Gold).
- A centralized feature layer is maintained to ensure consistency between on-line inference features and offline training datasets.

3. Analytical and Serving Boundary

- Anomaly detection is limited to travel-time deviations and congestion surges using Z-score statistics and Isolation Forest models.
- Results are served through a FastAPI-based REST layer and pushed to a real-time monitoring dashboard using Server-Sent Events (SSE).

1.3 Objectives

The primary goal of this thesis is to engineer a scalable, hybrid real-time analytics platform for detecting traffic anomalies. Specific objectives include:

- **Architecture Realization:** Implement a robust Medallion-based data lakehouse that supports incremental processing of Bronze (raw), Silver (cleaned), and Gold (aggregated) traffic datasets.
- **Low-Latency Propagation:** Achieve an end-to-end latency where anomaly signals are delivered to the serving layer within seconds of ingestion from the traffic API.

- Unified Detection Framework: Develop a dual-signal detector that reconciles statistical Z-scores with Isolation Forest inference to minimize False Positive Rates (FPR) in traffic monitoring.
- Stream-to-Batch Consistency: Ensure that historical traffic baselines stored in Iceberg tables are semantically consistent with the real-time features extracted during stream processing.
- Operational Observability: Provide a monitoring interface that displays real-time congestion indicators, ingestion lag, and route-level anomaly insights.

Chapter 2

RELATED WORK

This chapter delves into the existing literature and technological advancements that form the foundation of the UrbanPulse platform. It evaluates current methodologies in real-time urban monitoring, analyzes the architectural shift toward Data Lakehouses, and identifies the research gaps that necessitate a dual-signal anomaly detection framework.

2.1 Current Advancements

Traditional batch processing systems, such as those built on Apache Spark [1] or Hadoop-era ETL pipelines [2], excel at high-throughput historical analysis but introduce latency on the order of hours, making them unsuitable for time-critical urban incident detection. The Lambda Architecture [2] addressed real-time needs by running a parallel speed layer, yet at the cost of duplicated processing logic and operational complexity. The Kappa Architecture [3] simplified this by eliminating the batch layer entirely, unifying all processing through a single streaming pipeline using systems such as Apache Kafka [3] and Redpanda [4]. Separately, the Medallion Lakehouse architecture [5], powered by table formats such as Apache Iceberg [6], offers transactional integrity and schema evolution alongside a structured Bronze–Silver–Gold data lineage — addressing storage-layer concerns that are orthogonal to the choice of processing pattern. In the domain of anomaly detection, state-of-the-art approaches [7] leverage a combination of statistical baselines and unsupervised learning — such as Isolation Forest [8] — to handle the high dimensionality of city-scale telemetry, but these approaches have primarily been evaluated on offline, static benchmark datasets [7].

2.2 Research Gap

Despite these advancements, a significant *semantic gap* persists between raw telemetry ingestion and actionable intelligence. Most existing systems process traffic data in isolated batches, leading to high latency in incident response. There is a lack of integrated frameworks that ensure stream-to-batch consistency, where real-time anomaly signals are validated against historical neighbourhood rhythms stored in a transactional storage layer. Furthermore, systems that rely solely on static thresholds exhibit elevated False Positive Rates under low-variance, off-peak traffic conditions — a limitation that can be characterised as the *cold-start* problem. Critically, existing anomaly detection approaches are predominantly *static*: models are trained once on a fixed dataset and do not adapt as the underlying traffic distribution evolves, making them brittle in the face of seasonal shifts and infrastructure changes. A unified framework that couples continuous statistical baselining with periodic model retraining on accumulated historical data remains absent from the literature.

2.3 Theoretical Background

2.3.1 The Medallion Lakehouse Architecture

The Medallion pattern organises a Lakehouse into three progressively refined layers: Bronze stores raw ingested records without modification, Silver applies filtering and schema normalisation, and Gold aggregates data into business-level features ready for analytics and model training. Each layer boundary is a well-defined contract, enabling independent reprocessing of any tier without affecting upstream data. Figure 2.1 illustrates the conceptual data flow across these layers.

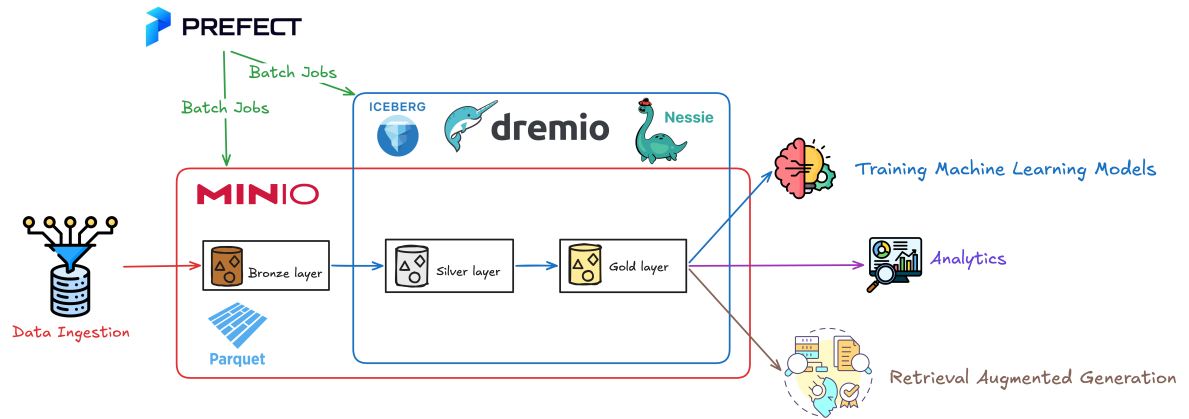


Figure 2.1: Conceptual overview of the Medallion Lakehouse architecture in UrbanPulse.

Stream-Batch Processing Patterns

Three canonical patterns govern how data pipelines balance latency against throughput [2]. Traditional Batch processing accumulates records and executes periodic bulk jobs, optimising for throughput at the cost of multi-hour latency. The Lambda Architecture [2] addresses this by running a parallel speed layer alongside the batch layer: the speed layer handles low-latency, per-event processing while the batch layer recomputes results on historical data; a unified serving layer merges both views at query time. The Kappa Architecture [3] simplifies Lambda by eliminating the batch layer entirely, routing all processing through a single streaming pipeline built on systems such as Apache Kafka or Redpanda [4]; the tradeoff is that historical recomputation must be replayed through the same stream, and the system maintains only one codebase. Table 2.1 summarises the data-flow characteristics of each pattern.

Table 2.1: Comparison of data processing architectural patterns by data-flow characteristics (✓ yes, ✗ no).

Characteristic	Traditional Batch [1]	Lambda Architecture [2]	Kappa Architecture [3]
Real-time speed layer (per-event processing)	✗	✓	✓
Batch layer (scheduled recomputation)	✓	✓	✗
Unified serving layer	✗	✓	✓

Analytical Storage Paradigms

Independently of which processing pattern is chosen, the storage layer must support concurrent writes from both the streaming and batch tiers, preserve raw data for re-processing, and allow ML workloads to access features without an intermediate export step. Three paradigms are commonly considered [5]. A Data Warehouse imposes a rigid schema-on-write model optimised for structured BI queries; raw data ingestion and direct ML access require an explicit ETL step, and storage formats are typically proprietary. A plain Data Lake stores data in open formats and supports direct ML access, but provides no transactional guarantees, making concurrent multi-writer scenarios unsafe and long-lived schema maintenance fragile. The Lakehouse paradigm merges the reliability of a warehouse with the openness of a data lake by adding an open table format layer above object storage. Table 2.2 summarises these paradigm-level capabilities.

Table 2.2: Comparison of analytical storage paradigms by data management capabilities (✓ yes, ✗ no).

Capability	Data Warehouse	Data Lake	Lakehouse (Iceberg)
Structured data support	✓	✓	✓
Raw data preservation	✗	✓	✓
ACID transactions	✓	✗	✓
Schema evolution	✗	✗	✓
ML training without data export	✗	✓	✓
Open storage format	✗	✓	✓
BI analytics support	✓	✗	✓

Table 2.2 motivates the choice of a Lakehouse over the two conventional alternatives. A Data Warehouse provides strong ACID guarantees and BI support but ingests only structured data, stores it in proprietary formats, and requires an explicit export step before ML training — precluding both the raw Bronze layer and direct feature access that UrbanPulse relies on. A plain Data Lake preserves raw data in open formats and allows direct ML access, but offers no ACID transactions or schema evolution, making concurrent writes from the streaming and batch layers unsafe and long-term schema maintenance fragile. The Lakehouse paradigm, instantiated via Apache Iceberg [6] on MinIO object storage with Project Nessie [9] as the versioned catalog, combines the reliability of a warehouse with the openness of a data lake, while additionally providing Git-like branch semantics for reproducible batch reprocessing.

2.3.2 Anomaly Detection Logic

UrbanPulse utilizes a dual-signal framework to identify traffic incidents. The statistical signal is calculated using the Z-score formula to detect immediate spikes in congestion ratio r relative to the moving average μ and standard deviation σ :

$$Z = \frac{r - \mu}{\sigma} \quad (2.1)$$

Simultaneously, an Isolation Forest [8] model operates on a 7-dimensional feature vector — four cyclical temporal encodings (sin/cos of hour of day and day of week) combined with three traffic ratio features — to identify non-recurrent patterns that simple statistics might miss. The model is formalized as:

$$f : \mathbb{R}^7 \rightarrow \mathbb{R} \quad (2.2)$$

$$\mathbf{x} = [\sin_h, \cos_h, \sin_{dow}, \cos_{dow}, r_{heavy}, r_{moderate}, n_{severe}] \quad (2.3)$$

where $f(\mathbf{x})$ is the anomaly score returned by the Isolation Forest's `decision_function` [8]. The score partitions the real line into two decision regions:

$$\text{label}(\mathbf{x}) = \begin{cases} \text{anomaly} & \text{if } f(\mathbf{x}) < 0 \quad (s \in [-1, 0)) \\ \text{normal} & \text{if } f(\mathbf{x}) \geq 0 \quad (s \in [0, +1]) \end{cases} \quad (2.4)$$

A score approaching -1 indicates a strong anomaly, while scores near $+1$ correspond to densely clustered, normal traffic observations.

Chapter 3

METHODOLOGY

This chapter presents the methodological framework underlying the design and implementation of UrbanPulse, a streaming-first urban intelligence platform for Ho Chi Minh City. The chapter is organized as follows. Section 3.1 establishes the functional and non-functional requirements through use case analysis. Section 3.2 describes the data acquisition and ingestion strategy. Section 3.3 details the multi-stage *Medallion Architecture* for stream processing and data transformation. Section 3.4 formalizes the dual-signal anomaly detection framework. Section 3.5 presents the *Retrieval-Augmented Generation* (RAG) pipeline for knowledge-enriched inference. Finally, Section 3.6 defines the evaluation framework and key performance indicators used in Chapter 4.

3.1 Requirement Analysis

The requirements of UrbanPulse are derived from the operational needs of municipal traffic management in a large Southeast Asian metropolitan area. Ho Chi Minh City presents a particularly demanding environment: a metropolitan population of approximately 8 million, with personal vehicles serving up to 85% of daily travel needs and peak-hour congestion patterns that shift substantially across zones, time of day, and meteorological conditions [10]. Existing monitoring approaches rely on periodic batch reporting, introducing temporal gaps of up to several hours between event occurrence and situational awareness [11]. UrbanPulse is designed to close this gap through continuous ingestion, real-time feature computation, and LLM-assisted root cause analysis.

3.1.1 Functional Requirements

The following functional requirements were identified through analysis of urban traffic management workflows:

1. FR-1 Continuous Data Ingestion: The system shall continuously acquire route-level traffic observations from a commercial mapping API and publish them to a durable, replayable message log with sub-minute freshness.
2. FR-2 Real-Time Feature Computation: For each ingested observation, the system shall compute and persist route-level statistical features (mean duration, heavy vehicle ratio, standard deviation) within 20 seconds of message arrival.
3. FR-3 Anomaly Detection: The system shall classify each route observation as anomalous or normal using at least two independent detection methods, and expose the results through a queryable REST API.
4. FR-4 Historical Analytics: The system shall maintain a versioned, queryable history of all traffic observations at Bronze, Silver, and Gold granularity, supporting point-in-time queries for reproducible analysis.

5. FR-5 Explainability: The system shall provide human-readable natural-language explanations of detected anomalies, grounded in historical context and meteorological conditions.
6. FR-6 Automated Alerting: The system shall deliver push notifications for high-confidence anomaly events with configurable per-route cooldown periods to suppress duplicate alerts.

3.1.2 Non-Functional Requirements

1. NFR-1 Latency: End-to-end ingestion lag (API response to persisted feature) shall not exceed 20 seconds at the 95th percentile under normal operating load.
2. NFR-2 Reproducibility: Any anomaly classification produced by the system shall be reproducible by querying the exact Lakehouse snapshot and MLflow model version active at the time of inference.
3. NFR-3 Fault Tolerance: The ingestion and feature computation pipelines shall tolerate transient service failures without data loss, through Kafka offset management and Postgres-based accumulator recovery.
4. NFR-4 Scalability: The architecture shall be horizontally scalable at the ingestion and serving layers without modification to the core processing logic.
5. NFR-5 Observability: All system components shall emit structured logs, metrics, and traces consumable by a centralised monitoring stack.

3.1.3 Use Case Description

Table 3.1 summarises the primary actors and their interactions with the UrbanPulse platform.

Table 3.1: Primary use cases and actors in UrbanPulse

Use Case	Actor	Trigger
Monitor live traffic map	Traffic operator	Continuous / event-driven push (sub-second after change)
Investigate anomaly	Traffic analyst	Alert notification received
Request root cause analysis	Traffic analyst	Manual query via dashboard
Review historical trend	Urban planner	Ad-hoc query
Receive anomaly alert	Duty manager	Automated Telegram notification
Retrain detection model	ML engineer	Scheduled (every 6 hours)

3.1.4 Use Case Diagram

Figure 3.1 illustrates the relationships between system actors and use cases. The *Traffic Operator* and *Traffic Analyst* interact primarily with the Serving API layer, whilst the *ML Engineer* interacts with the batch orchestration layer. The *External APIs* actor represents the automated data sources (VietMap and Open-Meteo) that trigger the ingestion pipeline.

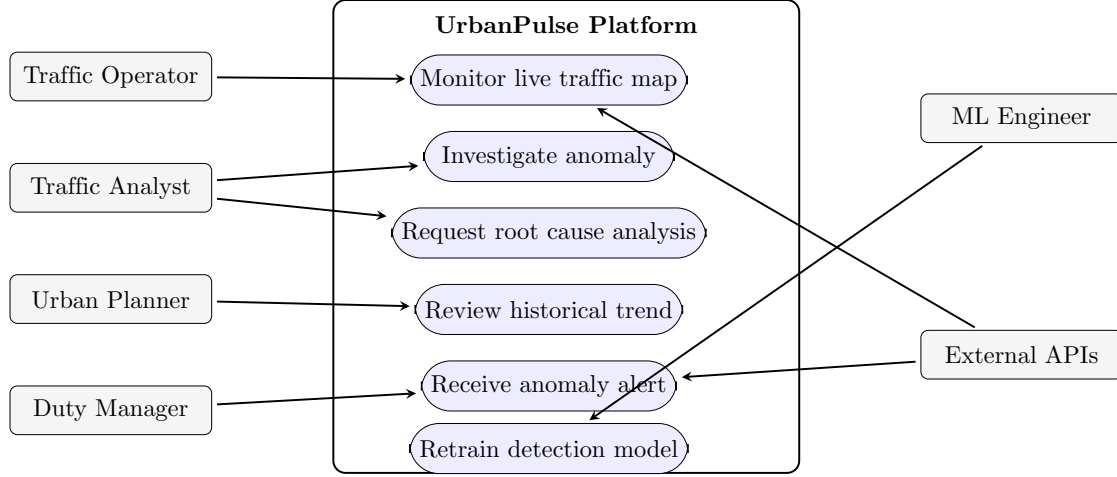


Figure 3.1: Use case diagram for the UrbanPulse platform.

3.1.5 System Overview

Figure 3.2 presents the high-level architecture of UrbanPulse. The platform is structured into four horizontal layers: the *Ingestion Layer*, the *Processing Layer*, the *Serving Layer*, and the *Inference Layer*. Data flows vertically from external APIs through the Redpanda message broker — a *Kafka*-compatible distributed streaming platform [12] — into two parallel consumer paths: a *durable archival path* that materialises data into a versioned Data Lakehouse, and a *real-time feature path* that writes statistical summaries to PostgreSQL for low-latency serving. The Serving Layer exposes a REST and Server-Sent Events (SSE) API surface. The Inference Layer augments API responses with LLM-generated explanations produced by a locally hosted Qwen2.5:3b model via Ollama [13].

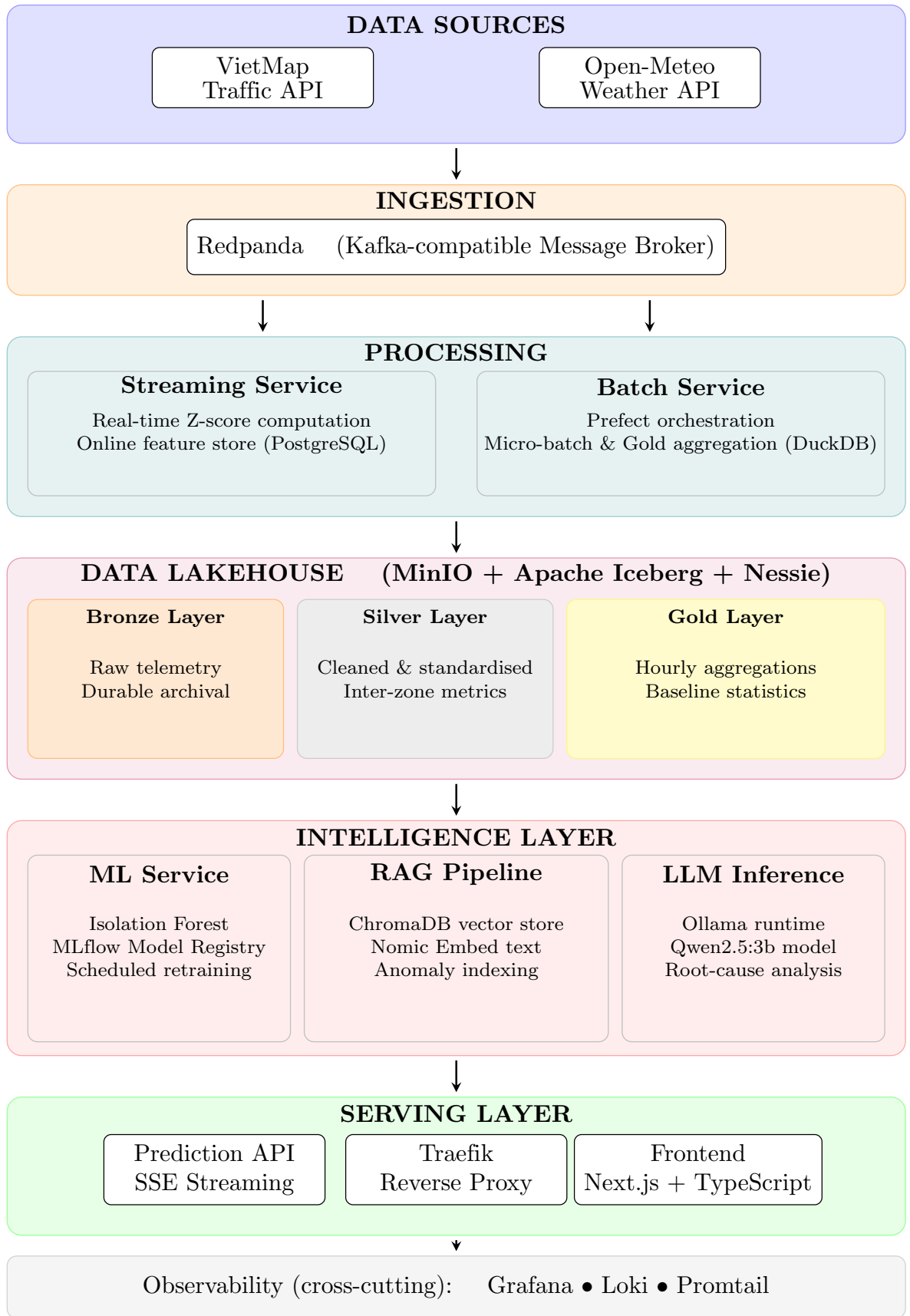


Figure 3.2: High-level system architecture of UrbanPulse.

3.2 Data Acquisition and Ingestion Strategy

Reliable, low-latency data acquisition is the foundation upon which all downstream processing depends. This section describes the two data sources integrated by UrbanPulse and the mechanism by which raw API responses are converted into a replayable event stream.

3.2.1 Traffic Telemetry — VietMap Directions API

Route-level traffic observations are acquired by periodically polling the VietMap Directions API [14], a commercial Vietnamese mapping provider with national road network coverage. The ingestion service iterates over the set of inter-zone routes defined in a canonical `routes.json` configuration file. Routes are generated from a `zones.json` specification that partitions Ho Chi Minh City into six functional zones: *Urban Core*, *Eastern Innovation*, *Northern Industrial*, *Southern Port*, *Western Peri-urban*, and *Southern Coastal*. Each route is expressed as a directed pair of origin and destination zone anchors. The route set is an operational subset of the full directed zone-pair space, selected to cover the major radial and cross-city corridors while keeping the dashboard and alert surface interpretable for thesis-scale evaluation.

For each route, a single API call returns the following fields: travel duration d_r (minutes), distance δ_r (metres), and a congestion breakdown comprising four segment-level ratios. Let S_r denote the set of road segments on route r . The congestion features are:

$$\text{heavy_ratio}_r = \frac{|\{s \in S_r : \text{level}(s) = \text{heavy}\}|}{|S_r|} \quad (3.1)$$

with analogous definitions for `moderate_ratior`, `low_ratior`, and `severe_segmentsr` (absolute count). A 10-second inter-request delay between consecutive API calls is enforced to comply with rate-limit constraints. A complete crawl cycle across all routes completes within the 5-minute polling interval.

It is important to clarify that VietMap operates as a polling-based source rather than a push-based event stream. The ingestion service bridges this gap by publishing each validated observation immediately to Redpanda via the Kafka producer protocol, effectively converting a batch API into a synthetic event stream. This pattern decouples the polling schedule from all downstream consumers: the pipeline behaves as a true streaming system regardless of source polling frequency [2].

Each API response is published to two Kafka topics: the raw VietMap response bytes are forwarded as-is to `vietmap-raw`; a transformed representation with extracted route metrics and computed congestion ratios is published to `traffic-route-bronze`. Both messages carry the same `ingest_ts` header — the Unix epoch millisecond timestamp recorded immediately before the API call — which serves as the reference point for end-to-end latency measurement in all downstream consumers.

3.2.2 Meteorological Context — Open-Meteo API

Hourly weather data is retrieved from the Open-Meteo API [15], a free, open-source meteorological service requiring no authentication. The `weather-ingestion` service polls Open-Meteo every 15 minutes (`_CYCLE_INTERVAL_S = 900`) for all six zone-specific coordinates independently (Table 3.2), publishing one `WeatherObservation` per zone

per cycle to Kafka. Retrieved variables include: temperature ($^{\circ}\text{C}$), precipitation (mm), rainfall (mm), wind speed (km/h), wind direction ($^{\circ}$), cloud cover (%), and WMO weather code.

Table 3.2: Zone-specific Open-Meteo query coordinates.

Zone ID	Name	Latitude	Longitude
zone1	Urban Core	10.7755 $^{\circ}$ N	106.6990 $^{\circ}$ E
zone2	Eastern Innovation	10.8667 $^{\circ}$ N	106.7793 $^{\circ}$ E
zone3	Northern Industrial	11.1031 $^{\circ}$ N	106.5496 $^{\circ}$ E
zone4	Southern Port	10.7630 $^{\circ}$ N	106.7722 $^{\circ}$ E
zone5	Western Peri-urban	10.6867 $^{\circ}$ N	106.5774 $^{\circ}$ E
zone6	Southern Coastal	10.5842 $^{\circ}$ N	107.0368 $^{\circ}$ E

Weather data is consumed in two operationally distinct modes:

1. Batch indexing mode: Zone-level observations flow from Kafka into the batch pipeline, which averages them into a single city-level record per hour and persists the result to the `gold.weather_hourly` Iceberg table. The RAG indexer then reads the trailing 7 days from this table (≈ 192 documents) and upserts them into the `external_context` ChromaDB collection (Section 3.5). A direct Open-Meteo API fallback is used only when `gold.weather_hourly` is not yet available (e.g. during initial bootstrap).
2. Live fetch mode: On each `/chat` API request, the serving layer performs a direct asynchronous HTTP fetch of current conditions using a single city-centre coordinate (10.7757 $^{\circ}$ N, 106.7009 $^{\circ}$ E), cached in-process for 15 minutes, and injects the result into the LLM system prompt.

Weather data is not used as an input to either anomaly detection model. Its role is exclusively contextual enrichment for the LLM inference layer described in Section 3.5.

3.3 Stream Processing and Medallion Architecture

Once published to Redpanda, traffic observations are consumed by two independent consumer groups operating in parallel — one for durable archival, one for real-time feature computation. This fan-out architecture, illustrated in Figure 3.3, ensures that neither path can block or slow the other. The following subsections describe each processing stage in the Medallion Architecture [5], a layered data quality pattern in which raw data is progressively refined through Bronze, Silver, and Gold tiers.

3.3.1 Bronze Layer — Durable Event Archival

The `apps/streaming` consumer buffers incoming Kafka messages in memory and flushes to MinIO object storage as Apache Parquet [16] files using a *dual-trigger flush policy*: a flush is issued when the buffer reaches 50 messages or when 30 seconds have elapsed since the last flush, whichever occurs first. This policy bounds the maximum number of small files written — which degrades downstream query performance [17] — and the maximum duration for which data remains unarchived in memory.

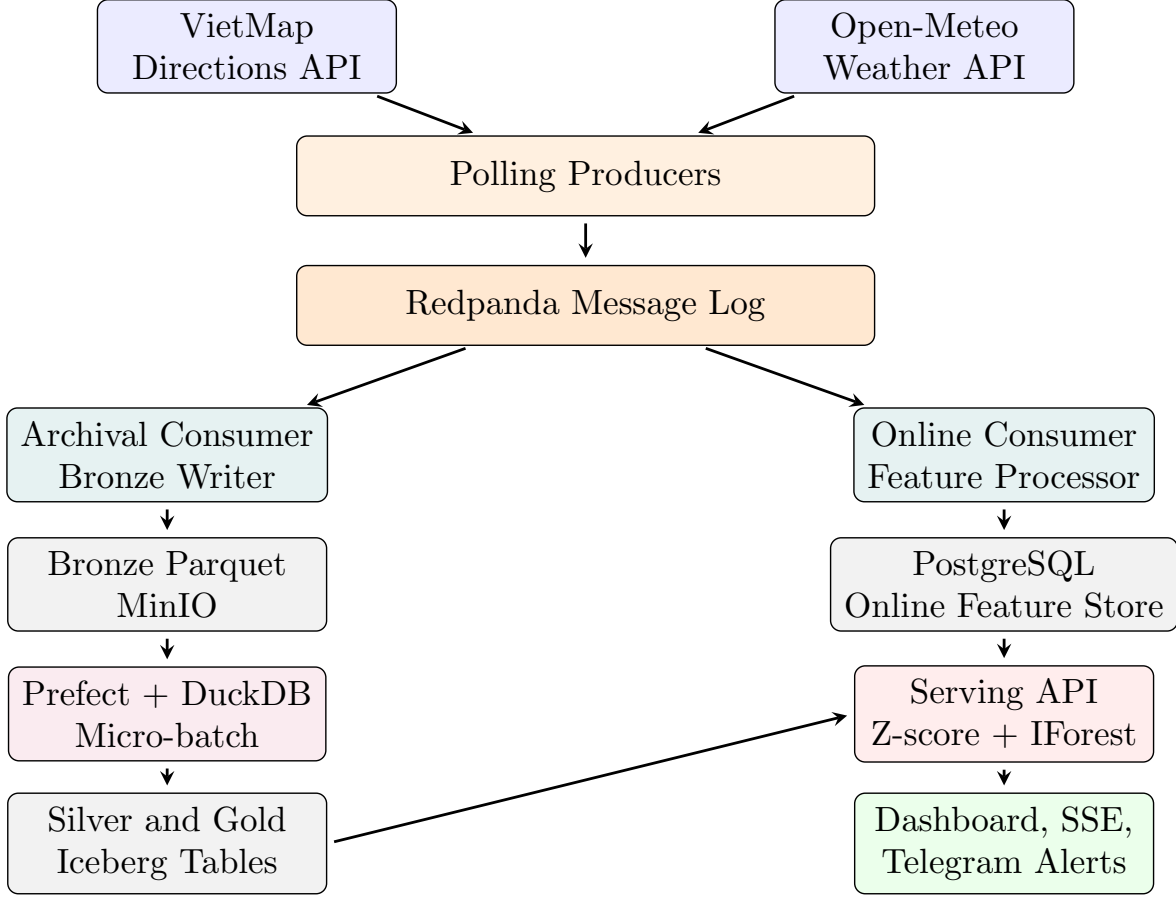


Figure 3.3: Dual-consumer data flow from Redpanda to Lakehouse and Online Feature Store.

Kafka offsets are committed only after a confirmed successful MinIO upload (`asynchronous=False`). If an upload fails, the consumer does not advance its offset; messages are re-played on restart, providing at-least-once delivery at the application level. Messages that cannot be processed after repeated attempts are routed to a *dead-letter queue* (DLQ) topic (`{topic}-dlq`) for independent inspection and recovery [18].

Parquet files are written with *Snappy* compression and use a Hive-style event-time partition scheme:

```
bronze/vietmap-raw/year=Y/month=M/day=D/hour=H/{uuid}.parquet
```

The partition key is derived from `timestamp_utc` embedded in the observation itself — not the time of writing. This *event-time partitioning* [19] ensures that late-arriving records are placed in the historically correct partition, preserving temporal integrity for all downstream aggregations.

3.3.2 Silver Layer — Incremental Micro-Batch Processing

Every 5 minutes, the `microbatch` Prefect [20] flow executes a DuckDB [21] scan over newly arrived Bronze Parquet files and promotes them to the Silver Apache Iceberg [22] table (`silver.traffic_route`), catalogued via Project Nessie [23].

It is necessary to distinguish this stage from *stateful stream processing* as implemented by systems such as Apache Flink [24] or Spark Structured Streaming [1]. The Silver layer uses an incremental micro-batch pattern: each Prefect run is a bounded,

stateless DuckDB job that reads only files written since the last high-watermark and appends to Iceberg atomically. This design was chosen deliberately: it eliminates the operational complexity of a long-running JVM streaming cluster while achieving Silver freshness within 5 minutes of event arrival, which is sufficient given that the sub-second real-time path is served by the Online Feature Store (Section 3.3.4).

Apache Iceberg provides *ACID-compliant snapshot isolation*: concurrent readers observe a consistent table snapshot whilst a new micro-batch commit is in progress. It additionally supports *schema evolution* (adding columns without invalidating existing readers) and *time-travel* (querying the table at any previous snapshot by ID or timestamp) [22]. These capabilities are foundational to the reproducibility requirement (NFR-2).

3.3.3 Gold Layer — Hourly Aggregation

Every hour, the hourly-gold Prefect flow aggregates the Silver table into `gold.traffic_hourly` — a pre-computed, per-hour summary. The Gold schema includes: `avg_heavy_ratio`, `avg_moderate_ratio`, `avg_duration_minutes`, and `max_severe_segments`. DuckDB reads the Silver Iceberg table via the PyIceberg [25] Arrow interface and writes results back to an Iceberg Gold table. The Gold layer serves two consumers: the *IsolationForest training pipeline* (Section 3.4.2) and the *RAG traffic pattern indexer* (Section 3.5.2).

3.3.4 Online Feature Store — Real-Time Postgres Path

In parallel with the archival path, the `apps/online` consumer maintains a live Online Feature Store in PostgreSQL (`online_route_features` table). For each Kafka message, the `OnlineFeatureProcessor` updates an in-memory hourly window for the corresponding route using Welford’s online algorithm [26] — a numerically stable, single-pass method for computing a running mean and standard deviation without retaining the full observation history.

The Welford accumulator state is persisted to PostgreSQL on every message via an UPSERT keyed on (`route_id`, `window_start`). On service restart, the processor reconstructs the accumulators from stored `mean`, `stddev`, and `count` via the inverse identity $M_2 = \sigma^2 \times (n - 1)$, allowing accumulation to resume from the exact point of interruption rather than resetting the hourly window. Algorithm 1 formalises this procedure.

Algorithm 1 $(\hat{\mu}, \hat{\sigma}, n) \leftarrow \text{WelfordUpdate}(x, \hat{\mu}_{\text{prev}}, M_{2,\text{prev}}, n_{\text{prev}})$

Require: x is the new heavy_ratio observation for route r .

Require: $\hat{\mu}_{\text{prev}}, M_{2,\text{prev}}, n_{\text{prev}}$ are the prior mean, sum of squared deviations, and count.

```

1:  $n \leftarrow n_{\text{prev}} + 1$ 
2:  $\delta \leftarrow x - \hat{\mu}_{\text{prev}}$ 
3:  $\hat{\mu} \leftarrow \hat{\mu}_{\text{prev}} + \delta / n$ 
4:  $\delta_2 \leftarrow x - \hat{\mu}$ 
5:  $M_2 \leftarrow M_{2,\text{prev}} + \delta \times \delta_2$ 
6: if  $n > 1$  then
7:    $\hat{\sigma} \leftarrow \sqrt{M_2 / (n - 1)}$ 
8: else
9:    $\hat{\sigma} \leftarrow 0$ 
10: end if
11: return  $(\hat{\mu}, \hat{\sigma}, n)$ 
```

Crash recovery. At startup, the processor queries PostgreSQL for the current hour’s stored `mean`, `stddev`, and `observation_count` for each route and reconstructs the Welford state as:

$$M_2^{\text{recovered}} = \hat{\sigma}^2 \times \max(n - 1, 0) \quad (3.2)$$

This ensures that a mid-hour service restart does not discard partially accumulated statistics, satisfying fault-tolerance requirement NFR-3.

3.4 Dual-Signal Anomaly Detection Framework

UrbanPulse employs two anomaly detection algorithms that are mathematically orthogonal by design: a *univariate threshold-based* detector operating on the real-time feature stream, and a *multivariate ensemble* detector trained on the historical Gold layer. Neither algorithm is a fallback for the other; they are fused at the serving layer to produce a three-tier confidence hierarchy. The rationale for combining both approaches follows from the observation that univariate methods miss complex joint anomalies, whilst ensemble methods lack the immediacy of per-message classification [7].

Specifically, the two detectors address complementary failure modes of urban traffic anomaly detection:

- **Z-Score (univariate):** Monitors a single variable — `heavy_ratio`, the fraction of heavily congested road segments — against a per-route, per-time-slot historical baseline. It detects *acute statistical spikes*: situations where the current window’s congestion intensity deviates sharply from what is normal at that hour on that route. Because it operates on a single dimension, it reacts within one Kafka message and is therefore suitable for *immediate* alerting. Its weakness is blindness to *multivariate* patterns: a route may appear normal on `heavy_ratio` alone whilst simultaneously exhibiting abnormally elevated `moderate_ratio` and `severe_segments` — a joint pattern invisible to univariate methods.
- **Isolation Forest (multivariate):** Operates on a 7-dimensional feature space $(h_{\text{avg}}, m_{\text{avg}}, s_{\text{max}}, h_{\text{sin}}, h_{\text{cos}}, d_{\text{sin}}, d_{\text{cos}})$ trained on the historical Gold layer. It detects *distributional outliers* — combinations of variables that are collectively rare given the route’s historical traffic rhythm. It captures anomalies that unfold gradually (e.g., a sustained moderate congestion surge on a normally free-flowing suburban route during an atypical weekday), which a univariate Z-score threshold may not exceed. Its weakness is latency: scores are computed in batch every 15 seconds from a model retrained every 6 hours, rather than per-message.

The conjunction rule (`both_anomaly` = Z-score $\wedge$ IForest) is therefore a deliberate precision-recall trade-off: it sacrifices recall on events detectable by only one signal in exchange for a high-confidence, low-false-positive alert channel suitable for automated Telegram notification.

3.4.1 Signal 1 — Z-Score Online Detector

Rationale and Role. Real-time anomaly detection requires a low-latency signal that responds instantly to traffic deviations. Z-score based on a rolling Welford baseline is

ideal: it runs in $O(1)$ memory per route and produces per-message scores, capturing immediate spikes before batch models can react [27, 28].

Input, Configuration, Output. The Online Service tracks a running mean of `heavy_ratio` for each route using Welford’s online algorithm, maintaining state (n, μ, M_2) in PostgreSQL. The batch baseline layer computes the 99th percentile of historical Z-scores ($p_{99,r}$) during the `baseline_learning` job. For each incoming observation, the Z-score is:

$$z_r^{(t)} = \frac{\bar{h}_r^{(t)} - \mu_r^{\text{base}}}{\sigma_r^{\text{base}}} \quad (3.3)$$

where $\bar{h}_r^{(t)}$ is the current Welford mean, and $\mu_r^{\text{base}}, \sigma_r^{\text{base}}$ are the batch baseline statistics. The adaptive threshold is:

$$\theta_r = \max(1.5, p_{99,r}), \quad p_{99,r} \stackrel{\text{def}}{=} \begin{cases} \hat{p}_{99,r} & \text{if history available} \\ 2.0 & \text{otherwise} \end{cases} \quad (3.4)$$

The floor of 1.5 prevents excessive sensitivity on low-variance routes; routes without sufficient history use a conservative fallback of 2.0. Because $p_{99,r}$ is learned from each route’s own Z-score distribution, high-variance routes naturally receive a higher threshold, reducing false positives without imposing an artificial ceiling. Classification is one-sided: only $z_r^{(t)} > \theta_r$ signals an anomaly (positive deviations indicate congestion; negative indicates free flow, which is not anomalous). The Z-score is stored in PostgreSQL’s `online_route_features` table as `duration_zscore` (schema legacy name retained for continuity) and exposed via the Serving Layer’s WebSocket API.

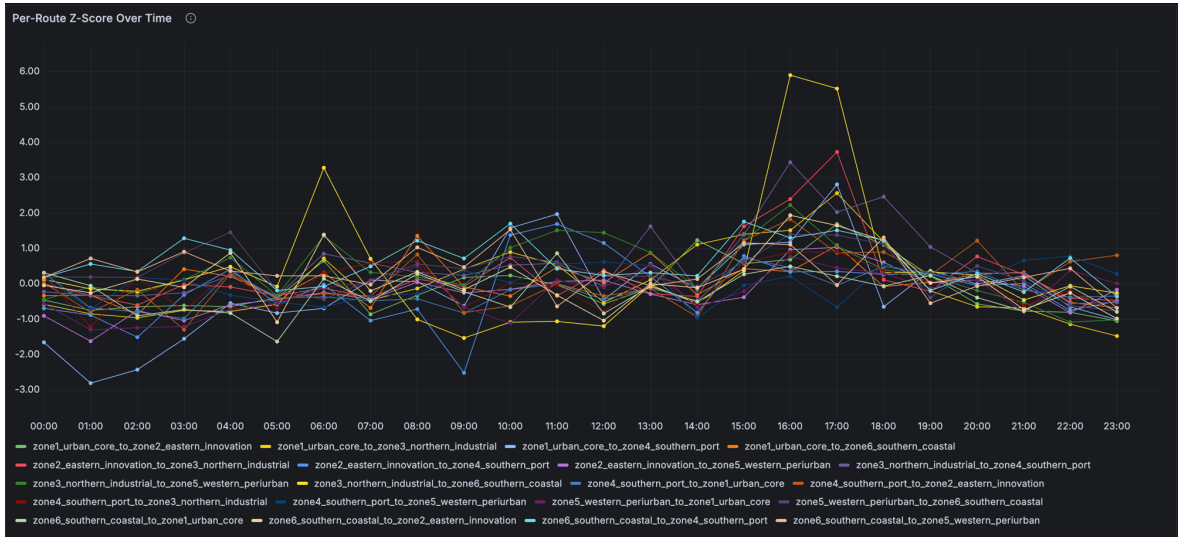


Figure 3.4: Per-route Z-score over a 24-hour observation window (Grafana). Each line represents one directed inter-zone route. Spikes above the dashed threshold θ_r trigger the Z-score anomaly signal.

3.4.2 Signal 2 — Isolation Forest Batch Detector

Rationale and Role. The dual-signal framework requires a second, independent detector to reconcile high-recall Z-score signals with high-precision multivariate outlier

detection. Isolation Forest [29] is well-suited for this role: it operates on a 7-dimensional feature space (congestion ratios + cyclical time encodings) without requiring labelled data or strong distributional assumptions, and its linear complexity in sample count enables 6-hourly retraining on homelab hardware [30].

Input, Configuration, Output. The feature vector per route-hour observation comprises three traffic metrics (avg heavy ratio, avg moderate ratio, max severe segments) and four cyclical time encodings (sine-cosine pairs for hour-of-day and day-of-week to ensure temporal circularity). An Isolation Forest model is trained every 6 hours on the Gold-layer corpus ($N \approx 1,600$ records per route) using Optuna Bayesian search to maximize the anomaly score gap.

$$\mathbf{x} = (h_{\text{avg}}, m_{\text{avg}}, s_{\text{max}}, \sin(2\pi \cdot \text{hr}/24), \cos(2\pi \cdot \text{hr}/24), \sin(2\pi \cdot \text{dow}/7), \cos(2\pi \cdot \text{dow}/7)) \quad (3.5)$$

The cyclical encodings (sine-cosine pairs) ensure temporal circularity: hour 23 and hour 0 are geometrically adjacent, preventing false ordinal distances that would mislead the model about traffic periodicity.

Table 3.3 reports the resulting configuration. The `contamination` value of 0.01 is consistent with an independent heavy-ratio Z-score anomaly rate estimate of 1.97% obtained by applying the statistical baseline rule to the same training corpus, providing cross-signal validation that genuine anomalous hours are rare in the pre-cutoff dataset.

Table 3.3: IsolationForest hyperparameters selected by Optuna TPE (50 trials $\times$ 20 routes; median aggregation).

Parameter	Search range	Default	Tuned
<code>contamination</code>	[0.01, 0.15]	0.05	0.01
<code>n_estimators</code>	{50, 100, ..., 500}	100	450
<code>max_samples</code>	[0.5, 1.0]	auto (≈ 0.17)	0.76
<code>max_features</code>	[0.5, 1.0]	1.0	0.73
Mean score gap: default 0.059 $\rightarrow$ tuned 0.083 (+40.7%, 20/20 routes improved).			

Figure 3.5 validates the tuned parameters against the default configuration across all 20 routes. The left panel plots tuned score gap against default score gap per route; all points lie above the diagonal, confirming improvement on every route. The right panel shows the distribution of score gap improvements (`tuned` $-$ `default`), with all values strictly positive (range +0.01 to +0.047, mean +0.023).

A lower `contamination` value raises the decision threshold, flagging only extreme outliers and biasing the detector towards precision; since dual-signal fusion (Section 3.4.3) handles recall via Z-score, IForest is tuned for high-confidence predictions. Every 6 hours, the model is retrained, registered in MLflow with the `@champion` alias, and loaded by the serving layer with a one-hour cache TTL.

3.4.3 Dual-Signal Fusion

The two signals are merged at the Serving Layer into a three-tier confidence hierarchy. Let $z_r \in \{0, 1\}$ denote the Z-score anomaly label and $f_r \in \{0, 1\}$ the IsolationForest anomaly label for route r . The fusion mapping is defined in Algorithm 2.

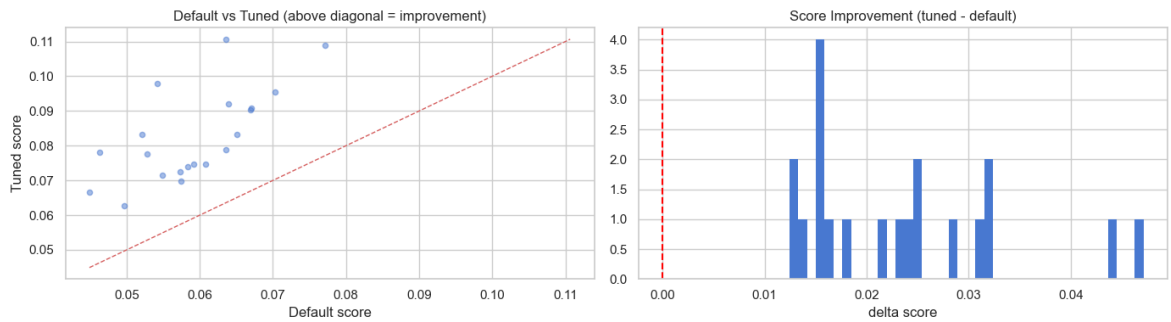


Figure 3.5: Optuna hyperparameter tuning validation across 20 routes. *Left*: tuned vs. default anomaly score gap per route (all points above the diagonal indicate improvement). *Right*: distribution of score gap delta (tuned – default); all 20 routes show strictly positive improvement (mean +0.023, max +0.047).

Figure 3.6 shows the MLflow experiment view for 'traffic-anomaly-detection'. The interface includes a search bar, tabs for 'Runs', 'Evaluation', 'Experimental', and 'Traces', and a table of runs. The table columns are Run Name, Created, F1, Dataset, Duration, Source, and Models. The runs are listed in descending order of creation time, showing 20 individual route models and several training runs.

Run Name	Created	F1	Dataset	Duration	Source	Models
per-route-train-20260419-1250	2 hours ago	-	-	2.2min	uvicorn	-
route-zone6_southern_coastal_to_zone5_western_perurban	2 hours ago	-	-	3.5s	uvicorn	iforest-zone6_southern_...
route-zone6_southern_coastal_to_zone4_southern_port	2 hours ago	-	-	3.4s	uvicorn	iforest-zone6_southern_...
route-zone6_southern_coastal_to_zone2_eastern_innovation	2 hours ago	-	-	3.4s	uvicorn	iforest-zone6_southern_...
route-zone6_southern_coastal_to_zone1_urban_core	2 hours ago	-	-	3.4s	uvicorn	iforest-zone6_southern_...
route-zone5_western_perurban_to_zone6_southern_coastal	2 hours ago	-	-	3.4s	uvicorn	iforest-zone5_western_p...
route-zone5_western_perurban_to_zone5_urban_core	2 hours ago	-	-	3.4s	uvicorn	iforest-zone5_western_p...
route-zone4_southern_port_to_zone5_western_perurban	2 hours ago	-	-	3.4s	uvicorn	iforest-zone4_southern_...
route-zone4_southern_port_to_zone3_northern_industrial	2 hours ago	-	-	3.4s	uvicorn	iforest-zone4_southern_...
route-zone4_southern_port_to_zone2_eastern_innovation	2 hours ago	-	-	3.4s	uvicorn	iforest-zone4_southern_...
route-zone4_southern_port_to_zone1_urban_core	2 hours ago	-	-	3.4s	uvicorn	iforest-zone4_southern_...
route-zone3_northern_industrial_to_zone6_southern_coastal	2 hours ago	-	-	3.4s	uvicorn	iforest-zone3_northern_i...
route-zone3_northern_industrial_to_zone5_western_perurban	2 hours ago	-	-	3.4s	uvicorn	iforest-zone3_northern_i...
route-zone3_northern_industrial_to_zone4_southern_port	2 hours ago	-	-	3.5s	uvicorn	iforest-zone3_northern_i...
route-zone2_eastern_innovation_to_zone5_western_perurban	2 hours ago	-	-	3.4s	uvicorn	iforest-zone2_eastern_in...
route-zone2_eastern_innovation_to_zone4_southern_port	2 hours ago	-	-	3.4s	uvicorn	iforest-zone2_eastern_in...
route-zone2_eastern_innovation_to_zone3_northern_industrial	2 hours ago	-	-	3.4s	uvicorn	iforest-zone2_eastern_in...
route-zone1_urban_core_to_zone6_southern_coastal	2 hours ago	-	-	3.4s	uvicorn	iforest-zone1_urban_cor...
route-zone1_urban_core_to_zone4_southern_port	2 hours ago	-	-	3.5s	uvicorn	iforest-zone1_urban_cor...
route-zone1_urban_core_to_zone3_northern_industrial	2 hours ago	-	-	3.4s	uvicorn	iforest-zone1_urban_cor...
route-zone1_urban_core_to_zone2_eastern_innovation	2 hours ago	-	-	3.4s	uvicorn	iforest-zone1_urban_cor...
per-route-train-20260419-0650	8 hours ago	-	-	2.2min	uvicorn	-
per-route-train-20260419-0050	14 hours ago	-	-	2.2min	uvicorn	-
per-route-train-20260418-1850	20 hours ago	-	-	2.2min	uvicorn	-
per-route-train-20260418-1250	1 day ago	-	-	2.2min	uvicorn	-

Figure 3.6: MLflow experiment view showing per-route IsolationForest training runs for the traffic-anomaly-detection experiment. Each row corresponds to one directed inter-zone route model.

Algorithm 2 $\text{tier}_r \leftarrow \text{SignalFusion}(z_r, f_r)$

Require: $z_r \in \{0, 1\}$ is the Z-score anomaly label.**Require:** $f_r \in \{0, 1\}$ is the IForest anomaly label.

```
1: if  $z_r = 1$  and  $f_r = 1$  then
2:   return Tier 1: both_anomaly ▷ Cross-validated; highest confidence
3: else if  $z_r = 1$  and  $f_r = 0$  then
4:   return Tier 2: zscore_anomaly ▷ Immediate spike; may be transient
5: else if  $z_r = 0$  and  $f_r = 1$  then
6:   return Tier 3: iforest_anomaly ▷ Multivariate pattern; not threshold-detectable
7: else
8:   return Normal
9: end if
```

Automated Telegram alerts are issued only for Tier 1 and Tier 3 signals, filtered through a 30-minute per-route cooldown managed in the `anomaly_alert_log` table. This threshold suppresses duplicate notifications during sustained congestion events whilst ensuring that any new independent anomaly episode triggers a fresh alert.

3.5 Knowledge-Enriched Inference via RAG

The anomaly detectors in Section 3.4 output reliable numerical signals (Z-score and Isolation Forest labels), but those outputs alone are not sufficient for operational decision-making. Traffic operators need an explanation of *why* congestion is occurring, not only whether an anomaly exists. UrbanPulse therefore applies Retrieval-Augmented Generation (RAG) [31] as an interpretation layer that converts dual-signal alerts into root-cause narratives grounded in historical traffic behaviour and current weather context.

3.5.1 Role in the System (Why)

The role of RAG in UrbanPulse is strictly application-facing: transforming raw anomaly alerts into actionable knowledge. When an event is flagged, the system should answer a practical question for city operators: *why is this route congested now?* In many cases, the explanation is compositional (for example, an unusual congestion spike coinciding with adverse weather during a peak-hour corridor). RAG is used to synthesise this multi-source context into concise natural language that can be consumed directly in dashboard workflows.

3.5.2 Implementation Pipeline (Input – Configuration – Output)

UrbanPulse implements RAG as a black-box inference pipeline with three operational stages:

1. Input (Retrieval): RAG is triggered for high-confidence events where both detection signals agree (`both_anomaly`). The serving layer retrieves traffic context from ChromaDB (recent anomaly events and historical route patterns) and, in parallel, fetches current meteorological context from Open-Meteo.

2. Configuration (Model and Prompting): Generation is executed by a local Qwen2.5:3b model served via Ollama [13, 32]. The local-LLM deployment is a deliberate infrastructure decision motivated by three constraints: municipal data privacy, predictable operating cost (no per-token external billing), and offline-capable deployment on city-controlled infrastructure. Retrieved traffic evidence and live weather variables are injected into a fixed prompt template together with numerical anomaly indicators (e.g., Z-score magnitude, congestion ratios, and route metadata).
3. Output (Generation): The LLM returns a concise root-cause explanation in natural language, streamed via Server-Sent Events (SSE) [33] and rendered directly in the operator interface (Grafana/UI) alongside the alert.

This implementation is intentionally concise: from anomaly trigger to human-readable explanation in a single retrieval-augmentation-generation flow, without additional iterative reasoning loops in the serving path.

3.5.3 Feedback Loop and Fine-Tuning Data Collection

Every LLM interaction — query string, retrieved chunks with similarity scores, and the full generated response — is persisted to the `rag_interaction_log` PostgreSQL table. The `/rca/feedback` endpoint accepts a thumbs-up (+1) or thumbs-down (−1) rating per interaction, stored in the `feedback` column. This produces a continuously growing, human-labelled dataset suitable for future *supervised fine-tuning* of the retrieval ranking or generation components [34], distinguishing UrbanPulse from systems that treat the LLM as a stateless inference endpoint.

3.6 Evaluation Framework

The methodologies described in the preceding sections are designed to satisfy the three operational pillars established in Section 3.1: *temporal integrity*, *reproducibility*, and *observability*. This section defines the specific metrics and measurement boundaries that Chapter 4 will report.

Experimental Protocol. The empirical evaluation follows a fixed protocol to ensure repeatable and comparable results. Traffic telemetry is collected continuously over a bounded observation window and retained at Bronze, Silver, Gold, and online-feature granularity. All experiments are executed on a single, fixed deployment environment; the hardware profile (CPU, memory, GPU, storage) and software stack (Docker, Python, Redpanda, PostgreSQL, MinIO, Iceberg/Nessie, MLflow, ChromaDB, Ollama) are recorded before each evaluation run.

The workload profile consists of three measurement modes: (i) normal-load run using production polling and all routes, (ii) stress run replaying messages at $2\times$ throughput to assess stability, and (iii) inference run issuing repeated requests to measure latency under steady-state conditions. Baseline comparison is performed against a batch-only variant where anomaly labels become available only after periodic aggregation. For each metric, measurements are computed from persisted logs or monitoring systems using p_{50} , p_{95} , and max statistics.

3.6.1 Temporal Integrity

All Bronze Parquet files are partitioned by event timestamp. End-to-end ingestion lag L_{e2e} is defined as:

$$L_{e2e}^{(r,t)} = t_{\text{feature}} - t_{\text{ingest_ts}} \quad (3.6)$$

where $t_{\text{ingest_ts}}$ is the Unix millisecond timestamp recorded immediately before the VietMap API call (embedded in the Kafka message header) and t_{feature} is the time at which the corresponding UPSERT completes in PostgreSQL. This metric is exposed per-route and aggregated at `/online/lag` as p_{50} , p_{95} , and `max`. The target for NFR-1 is $L_{e2e}^{p_{95}} < 20\text{ s}$.

3.6.2 Reproducibility

Two mechanisms jointly ensure reproducibility. First, the Apache Iceberg time-travel API allows any Silver or Gold table to be queried at any historical snapshot by ID or timestamp — meaning the exact data state at the time of any anomaly classification can be reconstructed. Second, MLflow records the full provenance of every Isolation-Forest training run: hyperparameters (`contamination`, `n_estimators`, `max_samples`, `max_features`, `random_state`, `training_cutoff`), feature schema, and training data time range; therefore, each inference can be replayed against the same model version and data snapshot.

3.6.3 Performance Metrics

The following metrics summarise the system’s ability to satisfy the operational pillars:

Table 3.4: Performance metrics: temporal integrity, reproducibility, and system performance

KPI	Definition	Target	Measurement point
Ingestion lag p_{95}	$t_{\text{feature}} - t_{\text{ingest_ts}}$	$< 20\text{ s}$	<code>/online/lag</code>
Microbatch freshness	Bronze write $\rightarrow$ Silver commit	$< 5\text{ min}$	Prefect task duration
IForest scoring latency	Background scorer cycle duration (15 s interval)	$< 5\text{ s}$	Serving API logs

By integrating periodic high-velocity API polling with an incrementally processed, versioned Data Lakehouse and two independent, orthogonal anomaly detectors, the methodology presented in this chapter provides a rigorous foundation for evaluating UrbanPulse’s ability to maintain low-latency, reproducible situational awareness under real-world metropolitan workloads in Ho Chi Minh City.

Chapter 4

PROTOTYPING

This chapter documents the prototyping phase of the UrbanPulse platform, covering the technology selection rationale, containerised infrastructure design, core data schema decisions, and the incremental validation strategy used before full implementation began.

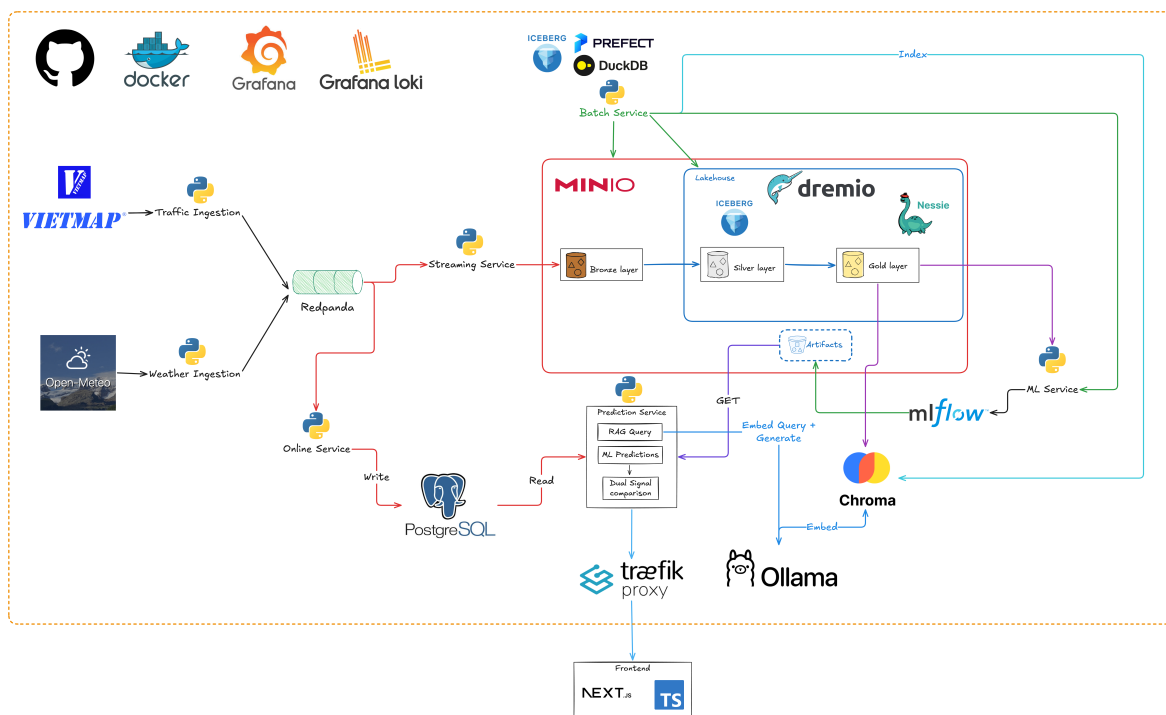


Figure 4.1: UrbanPulse complete system architecture: data ingestion from VietMap and Open-Meteo via Redpanda, medallion lakehouse (Bronze/Silver/Gold) on MinIO with Iceberg and Nessie, Online Service writing to PostgreSQL, Batch and ML Services tracked by MLflow, RAG pipeline via ChromaDB and Ollama, and Next.js frontend served through Traefik.

4.1 Technology Stack Selection

The UrbanPulse prototype is built on a curated stack of open-source components. Each technology was evaluated against three criteria: *(i)* compatibility with Python 3.12 and the `uv` monorepo toolchain, *(ii)* support for streaming-first workloads, and *(iii)* availability as a self-hostable Docker image suitable for homelab deployment. Table 4.1 summarises the chosen components and their roles.

4.1.1 Message Broker: Redpanda over Apache Kafka

Redpanda [12] was selected over vanilla Apache Kafka for two practical reasons. First, Redpanda eliminates the ZooKeeper dependency, reducing the container count by two

Table 4.1: UrbanPulse technology stack and selection rationale.

Component	Technology	Version	Rationale
Message Broker	Redpanda	v25.3.10	Kafka-compatible; no ZooKeeper; lower memory
Object Storage	MinIO	latest	S3-compatible API; local deployment
Table Format	Apache Iceberg	0.9.x	Schema evolution; time-travel; partition pruning
Catalog	Project Nessie	0.99.0	Git-like branching; REST API
Online Store	PostgreSQL	16	UPSERT semantics; asyncpg for async serving
Vector Store	ChromaDB	latest	Embedded; nomic-embed compatible
Orchestration	Prefect	3.x	Task retries; flow scheduling; UI dashboard
ML Tracking	MLflow	2.21.3	Experiment logging; model registry
LLM Runtime	Ollama	latest	Local GPU inference; Qwen2.5:3b
Serving	FastAPI	0.115.x	Async; SSE support; Pydantic v2
Reverse Proxy	Traefik	v2.11	Automatic TLS; label-based routing
Monitoring	Grafana + Loki	11.6.0 + 3.4.2	Log aggregation; dashboard
Frontend	Next.js 15 + TypeScript	15	App Router; real-time SSE client

services. Second, its *single-binary* architecture yields lower memory consumption, which is critical for the homelab Intel i5 deployment environment. The Kafka-wire compatibility ensures that all `confluent_kafka` Python clients work without modification.

4.1.2 Storage: Iceberg + Nessie + MinIO

Raw Parquet files in MinIO constitute the Bronze layer. Promoting data to Silver and Gold requires a table format that supports (a) transactional writes from micro-batches, (b) schema evolution without full rewrites, and (c) time-travel for reprocessing. Apache Iceberg [22] satisfies all three. Project Nessie [23] provides the REST catalog, enabling branching and atomic commits without locking MinIO objects directly.

4.1.3 LLM: Ollama with Qwen2.5:3b

The RAG pipeline requires a model that fits within the homelab GPU budget (NVIDIA GTX 1060, 6 GB VRAM). Qwen2.5:3b [13], served via Ollama, achieves an average time-to-first-token of under one second while producing coherent 2–3 sentence traffic explanations in both English and Vietnamese. The *RAG-over-fine-tuning* approach is justified by the need for always-current context: retraining on new anomaly events would be prohibitively expensive on the available hardware.

4.2 Service Topology Design

The platform is decomposed into seven microservices, each running as an independent Docker container. Services communicate exclusively through two channels: the Redpanda broker (event streams) and PostgreSQL (materialised feature store). No direct service-to-service HTTP calls exist in the hot path, ensuring fault isolation.

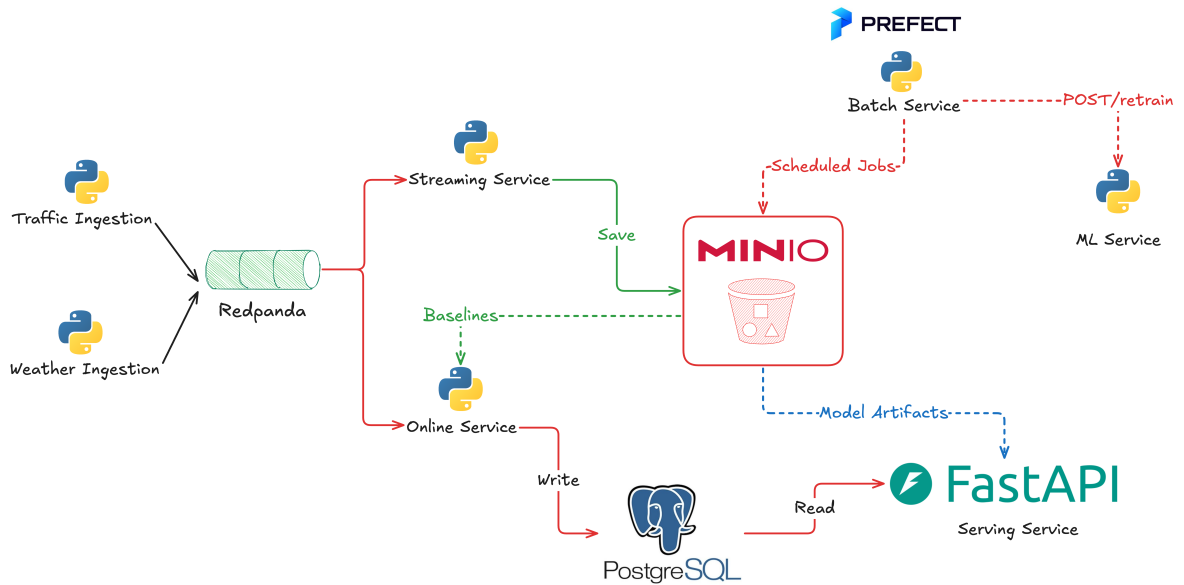


Figure 4.2: UrbanPulse service topology

4.3 Data Schema Design

4.3.1 Kafka Message Schema

Every traffic event published to Redpanda carries the raw VietMap Directions API response as its message body, with route context attached as Kafka headers. This separation keeps the payload at full source fidelity while making routing metadata available to consumers without deserialising the body.

- Key: `route_id` — unique string identifier of the inter-zone route (e.g., `zone1_urban_core_to` used by Kafka for partition affinity).
- Body: the raw JSON response from the VietMap Directions API, preserved verbatim. The relevant nested structure includes `paths[0].time` (travel time in milliseconds), `paths[0].distance` (metres), and `paths[0].annotations.congestion` — an array of segment-level congestion labels ("low", "moderate", "heavy", "severe", or "unknown").

Three Kafka headers are attached at publish time:

- `route_id`: mirrors the message key for consumers that read headers without inspecting the body.
- `timestamp_utc`: ISO-8601 timestamp of the API poll instant, recorded by the ingestion service before the HTTP request is issued.
- `ingest_ts`: the same instant expressed as a Unix millisecond epoch, used by downstream consumers to compute end-to-end ingestion lag.

No schema validation or field extraction is performed by the ingestion service; transformation is deferred to the batch layer, where it can be revised without reprocessing raw data.

4.3.2 Iceberg Table Schemas

Three Iceberg tables are provisioned in MinIO via Nessie catalog:

1. Silver (`silver.traffic_route`): 10-column schema with one row per API observation. Partitioned by `day(timestamp_utc)`. A `source` column records the VietMap API endpoint version.
2. Gold (`gold.traffic_hourly`): 11-column hourly aggregation keyed by (`route_id`, `hour_utc`). Includes `avg_heavy_ratio`, `avg_moderate_ratio`, `avg_duration_minutes`, and `p95_duration_minutes`. Partitioned by `day(hour_utc)`.
3. Baseline (`gold.traffic_baseline`): per-route, per-hour-of-day, per-day-of-week statistical summary used by the Online Service Z-score computation.

4.3.3 Weather Schema

The `WeatherObservation` Pydantic model carries hourly meteorological readings per zone: `temperature_c`, `precipitation_mm`, `wind_speed_kmh`, `cloud_cover_pct`, and `wmo_code`. These are published by the `weather-ingestion` service to a separate Kafka topic and consumed exclusively by the Batch Service, which persists a city-level hourly average to the `gold.weather_hourly` Iceberg table and indexes the trailing 7 days into the `external_context` ChromaDB collection for RAG retrieval. Weather data is not an input to either anomaly detector; its role is limited to contextual enrichment of LLM-generated root-cause explanations (Section 3.5).

4.3.4 PostgreSQL Online Store

The table `online_route_features` persists the current hourly window state for every route, with primary key (`route_id`, `window_start`). It serves two purposes: (a) it is the low-latency read target of the Serving API, and (b) it acts as a crash-recovery checkpoint — on restart, the Online Service reconstructs Welford accumulators from stored `mean` and `stddev` values via the identity $M_2 = \sigma^2 \times (n - 1)$.

4.4 End-to-End Data Flow Through the Medallion Layers

The three schema tiers described above form a progressive refinement pipeline. Table 4.2 summarises the transformation applied at each layer, the responsible service, and the latency guarantee relative to the originating API poll.

Table 4.2: Medallion data flow: transformation, ownership, and latency per layer.

Layer	Format	Transformation	Owner	Freshness
Bronze	Parquet (MinIO)	Raw JSON $\rightarrow$ typed columns, Snappy-compressed	Streaming Service	≤ 30 s
Silver	Iceberg table (Nessie)	Dedup, type-coerce, partition by day	Batch Service (microbatch)	≤ 5 min
Gold	Iceberg table (Nessie)	Hourly aggregation (avg, p95) per route	Batch Service (hourly)	≤ 1 h
Baseline	Iceberg table (Nessie)	Per-route/hour/dow mean+stddev for Z-score	Batch Service (retrain)	≤ 6 h
Online	PostgreSQL	Welford rolling window, live Z-score signal	Online Service	≤ 20 s

A critical design property is that the Bronze layer is never modified after writing: it is an immutable, append-only archive. All refinements — deduplication, type coercion,

aggregation — are performed by downstream layers that *read* Bronze and *write* Silver or Gold as new Iceberg snapshots. This separation means that any downstream corruption can be fully repaired by replaying the Bronze-to-Silver micro-batch from any historical Kafka offset, without re-polling the VietMap API.

4.4.1 Data Versioning with Nessie and Iceberg

Project Nessie acts as a Git-like catalog for Iceberg tables. Each micro-batch commit to `silver.traffic_route` or `gold.traffic_hourly` creates a new *snapshot* in Nessie, identified by a monotonically increasing snapshot ID and a wall-clock timestamp. This versioning provides two operational guarantees:

1. Reproducibility (NFR-2): Any anomaly classification can be re-run against the exact data state at the time of inference by querying the Iceberg table **AS OF** the snapshot ID recorded in the MLflow run. This is the same time-travel mechanism used in the evaluation framework (Section 3.6).
2. Stream-batch consistency: Because the batch pipeline commits to Iceberg atomically (all rows of a micro-batch in a single snapshot), any downstream reader always observes a *complete*, consistent view of the Gold table — never a partial write mid-aggregation. This eliminates the *phantom read* problem that arises when streaming systems read from mutable database tables. The concrete consumer of this guarantee is the baseline-loading mechanism described in Section 5.4.

4.5 Infrastructure Prototyping

The entire platform is containerised using Docker Compose with a three-file layered override pattern:

- `docker-compose.base.yaml` — shared service definitions (Redpanda, MinIO, Nessie, PostgreSQL, ChromaDB, MLflow, Prefect, Ollama).
- `docker-compose.dev.yaml` — development overlay: exposes ports locally, mounts source code volumes, uses `host.docker.internal` for the Ollama endpoint.
- `docker-compose.prod.yaml` — production overlay: adds Traefik reverse proxy with Let's Encrypt TLS, removes exposed ports, uses named volumes at `/opt/urban-pulse/` on the host.

The `Makefile` provides validated entry points (`make dev`, `make prod`, `make bootstrap`, `make train`) so that the multi-container stack can be brought up or torn down deterministically. The `bootstrap` target seeds Redpanda topics, MinIO buckets, and Prefect deployment schedules in a single pass before live traffic is processed.

Chapter 5

IMPLEMENTATION

This chapter details the implementation of each UrbanPulse microservice, tracing the data path from raw API polling through streaming ingestion, lakehouse persistence, machine learning inference, and finally the serving layer that delivers results to the frontend.

5.1 Ingestion Service

UrbanPulse operates two independent ingestion services, each responsible for a distinct data source: the Traffic Ingestion Service polls the VietMap Directions API for road-level congestion telemetry, and the Weather Ingestion Service polls the Open-Meteo API for hourly meteorological observations across all six HCMC zones. Both services publish raw records to dedicated Redpanda topics and share no state, allowing them to be deployed, scaled, and restarted independently.

5.1.1 Traffic Ingestion Service

The Traffic Ingestion Service is a long-running Python process that polls the VietMap Directions API for 20 predefined inter-zone routes across Ho Chi Minh City. After each full crawl cycle completes, the service sleeps for 30 seconds before beginning the next cycle; routes are polled sequentially with a 10-second inter-request delay to respect the API rate limit. With 20 routes and a mean API round-trip of approximately two seconds, each route receives one observation roughly every four to five minutes. Each route is defined in `routes.json` by an origin and destination anchor point (WGS-84 latitude/longitude pair) together with a human-readable zone label.

VietMap API Integration

The `sources/vietmap.py` module constructs a REST request to the `/api/route/v3` endpoint, requesting `congestion` annotations per road segment. Crucially, no field extraction or schema validation is performed at this stage: the API response is returned verbatim as a Python dictionary alongside the poll timestamp.

```

1 def fetch_route_raw(route_id, origin, destination,
2                     origin_anchor, destination_anchor, api_key):
3     url = (f"{_BASE_URL}?apikey={api_key}"
4           f"&point={origin_anchor[0]},{origin_anchor[1]}"
5           f"&point={destination_anchor[0]},{destination_anchor[1]}"
6           f"&points_encoded=false&vehicle=car"
7           f"&annotations=congestion")
8     polled_at_ms = int(time.time() * 1000)
9     timestamp_utc = datetime.now(timezone.utc)
10    resp = requests.get(url, timeout=30)
11    resp.raise_for_status()
12    return resp.json(), polled_at_ms, timestamp_utc

```

Listing 5.1: Raw API fetch in `sources/vietmap.py`.

The raw response is published to the Redpanda topic `vietmap-raw` with the message body set to the JSON-encoded API payload and route context carried entirely in Kafka headers:

```

1 def publish(self, route_id, polled_at_ms, timestamp_utc, raw_response)
2     :
3     payload = json.dumps(raw_response, ensure_ascii=False).encode()
4     headers = {
5         "route_id": route_id.encode(),
6         "timestamp_utc": timestamp_utc.isoformat().encode(),
7         "ingest_ts": str(polled_at_ms).encode(),
8     }
9     self._producer.produce(
10         TRAFFIC_TOPIC, key=route_id,
11         value=payload, headers=headers)

```

Listing 5.2: Kafka publish in `publishers/kafka.py`.

This design confines the ingestion service to a single responsibility: *collect and forward*. All transformation logic — congestion ratio computation, duration derivation, anomaly scoring — is deferred to downstream consumers, ensuring that the raw event record can always be reprocessed if business logic changes.

Sequential Route Polling

The `orchestrator.py` module iterates over all 20 routes sequentially, inserting a 10-second sleep between consecutive API calls. Failed requests are caught per-route, logged, and skipped; the process continues with the remaining routes and retries the failed route automatically in the next cycle. This sequential design was preferred over concurrent polling to avoid saturating the VietMap API quota and to simplify error attribution to individual routes.

5.1.2 Weather Ingestion Service

The Weather Ingestion Service polls the Open-Meteo Forecast API every 15 minutes for all six HCMC metropolitan zones. Each polling cycle issues one HTTP request per zone, retrieves hourly observations for the past 24 hours plus the next 24 hours, and selects the most recently *completed* hour — defined as the latest hour whose `hour_utc` is strictly before the current UTC clock hour and whose temperature reading is non-null. This selection strategy avoids publishing incomplete hourly aggregates that the Open-Meteo API may return for the in-progress hour.

5/4/2026, 1:38:10 AM

zone4_southern_port_to_zone1_urban_core

TEXT - 39 B

{
"license": "vietmap",
"code": "OK",
"paths": [{"d...
JSON - 17.6 kiB

...

Partition

Offset

Key

Value

Headers

Compression

Transactional

0

8123

Text (39 B)

Json (17.6 kiB)

3

uncompressed

false

Key (39 B)

Value (17.6 kiB)

Headers (3)

KEY

VALUE

ENCODING

>

ingest_ts

"1777833490649"

text

>

route_id

"zone4_southern_port_to_zone1_urban_core"

text

>

timestamp_utc

"2026-05-03T18:38:10.649873+00:00"

text

Download Record

>

5/4/2026, 1:38:20 AM

zone4_southern_port_to_zone2_eastern_inno...
TEXT - 47 B

{
"license": "vietmap",
"code": "OK",
"paths": [{"d...
JSON - 16.8 kiB

>

5/4/2026, 1:38:31 AM

zone4_southern_port_to_zone3_northern_ind...
TEXT - 48 B

i

Message size exceeds the display limit.

>

5/4/2026, 1:38:41 AM

zone4_southern_port_to_zone5_western_peri...
TEXT - 46 B

{
"license": "vietmap",
"code": "OK",
"paths": [{"d...
JSON - 14.5 kiB

Figure 5.1: Redpanda Console: `vietmap-raw` topic confirming continuous ingestion across the observation period.

The six zones and their WGS-84 coordinates are hardcoded in `sources/openmeteo.py` to match the zone geometry used by the traffic ingestion service:

Zone ID	Description
<code>zone1_urban_core</code>	Urban Core
<code>zone2_eastern_innovation</code>	Eastern Innovation District
<code>zone3_northern_industrial</code>	Northern Industrial Zone
<code>zone4_southern_port</code>	Southern Port Area
<code>zone5_western_periurban</code>	Western Peri-urban
<code>zone6_southern_coastal</code>	Southern Coastal

Seven hourly variables are requested from the API: `temperature_2m`, `precipitation`, `rain`, `wind_speed_10m`, `wind_direction_10m`, `cloud_cover`, and `weather_code`. The raw WMO integer weather code is translated to a human-readable description using a static lookup table, and wind direction degrees are mapped to an 8-point compass name. A zone that returns no valid observation is skipped silently, so a single API failure does not block the remaining five zones in the same cycle.

Each observation is serialised to JSON via the `WeatherObservation` Pydantic model and published to the `weather-hcmc-bronze` Redpanda topic. The message key is set to `location_id` (ensuring all observations for a given zone land in the same partition) and the `ingest_ts` header carries the Unix millisecond timestamp of the polling cycle start. A complete cycle produces exactly six messages — one per zone — before the service sleeps for 15 minutes (`_CYCLE_INTERVAL_S = 900`).

5.2 Streaming Service

The Streaming Service runs a single consumer group (`streaming-group`) that subscribes to two topics in parallel: `vietmap-raw` for traffic telemetry and `weather-hcmc-bronze`

for meteorological observations. Each topic is handled by a dedicated processor instance; the main event loop dispatches each incoming message to the correct processor based on its topic, and the dual-trigger flush policy (size or time threshold) is checked independently for each processor on every idle poll cycle.

5.2.1 Traffic Processor

The traffic processor (`processors/traffic.py`) treats each Kafka message as an opaque event: the message body — the raw VietMap API response — is stored verbatim as a JSON string, while route context (`route_id`, `timestamp_utc`, `ingest_ts`) is extracted from Kafka headers without touching the payload. No deserialisation or schema validation is performed; this preserves full source fidelity and eliminates the risk of validation failures causing data loss.

Each message is appended to an in-memory buffer as a four-tuple:

```
(raw, route_id, timestamp_utc, ingest_ts)
```

The buffer is flushed to MinIO as a Snappy-compressed `PyArrow` Table via `sinks/minio.py` using a dual-trigger flush policy: a flush occurs when the buffer reaches 500 records or when 30 seconds have elapsed since the last flush, whichever comes first. This policy bounds both the number of small files written to MinIO and the maximum duration for which ingested data remains unarchived in memory. Files are written under a time-partitioned path:

```
s3://urban-pulse/bronze/vietmap-raw/  
year={Y}/month={M}/day={D}/hour={H}/{uuid}.parquet
```

The partition key is derived from the `ingest_ts` header of the first record in each flush batch, ensuring that files land in the partition corresponding to when they were written rather than the API poll time, which can diverge by several seconds under high API latency. The resulting Parquet files carry four columns: `raw` (string), `route_id` (string), `timestamp_utc` (string), and `ingest_ts` (int64).

At-least-once delivery is enforced through offset commit ordering: Kafka consumer offsets are committed only after a successful MinIO upload, so a process restart will replay all uncommitted messages from the last committed offset. On a processing exception, the buffer is not cleared (preventing data loss) and the triggering message is forwarded on a best-effort basis to the dead-letter topic `vietmap-raw-dlq` for operational inspection.

5.2.2 Weather Processor

The weather processor (`processors/weather.py`) handles messages on the `weather-hcmc-bronze` topic. Unlike the traffic processor, each message is a fully structured `WeatherObservation` Pydantic model deserialised from JSON on ingestion, since the Open-Meteo API response schema is stable and known ahead of time. The processor therefore validates and deserialises each record immediately rather than storing the raw payload.

The flush policy uses the same 30-second time threshold but a much smaller buffer cap of 10 records, reflecting the lower message rate of the hourly weather feed compared to the continuous VietMap traffic stream. Flushed records are written as Snappy-compressed Parquet files to a path partitioned by the `hour_utc` field of the observation:


```
s3://urban-pulse/bronze/weather-hcmc-bronze/
year={Y}/month={M}/day={D}/hour={H}/{uuid}.parquet
```

Each file stores twelve columns per record: `location_id`, `hour_utc` (UTC timestamp), `temperature_c`, `precipitation_mm`, `rain_mm`, `wind_speed_kmh`, `wind_direction_deg`, `wind_direction_name`, `cloud_cover_pct`, `weather_code` (WMO integer), `weather_desc` (human-readable string), and `source`. The same at-least-once offset commit semantics and dead-letter forwarding to `weather-hcmc-bronze-dlq` apply as for the traffic processor.

5.3 Online Service

The Online Service runs a second consumer group (`online-features-group`) on the same `vietmap-raw` topic in parallel with the Streaming Service, implementing a stateful Welford online algorithm [26] for per-route rolling statistics that feed the real-time anomaly signal.

5.3.1 Message Ingestion

Each Kafka message carries the raw VietMap API response as its body, with route identity and timing in the headers (`route_id`, `timestamp_utc`, `ingest_ts`). The processor reads `route_id` and `ingest_ts` directly from the headers and decodes the JSON body to extract the first path entry:

- `paths[0].time` — travel duration in milliseconds, converted to *duration_minutes*;
- `paths[0].annotations.congestion` — per-segment congestion labels used to compute heavy, moderate, and low ratios and the count of severe segments.

End-to-end ingestion lag is computed as $\Delta t = t_{\text{now}} - \text{ingest_ts}$ and stored in `last_ingest_lag_ms` for observability.

5.3.2 Welford Accumulator

For each route, a `RouteWindow` object maintains the Welford state $(n, \bar{x}, M_2)$ where n is the observation count, $\bar{x}$ is the running mean of travel duration, and $M_2 = \sum (x_i - \bar{x})^2$. On each new observation x :

$$n \leftarrow n + 1 \tag{5.1}$$

$$\delta = x - \bar{x} \tag{5.2}$$

$$\bar{x} \leftarrow \bar{x} + \delta/n \tag{5.3}$$

$$M_2 \leftarrow M_2 + \delta \cdot (x - \bar{x}) \tag{5.4}$$

$$\sigma = \sqrt{M_2/(n-1)} \quad (n > 1) \tag{5.5}$$

This formulation is numerically stable and requires $O(1)$ memory regardless of observation history [26]. Separate accumulators are maintained for *duration_minutes* (Welford) and for *heavy_ratio*, *moderate_ratio*, *low_ratio* (running sums), since the congestion ratios do not need a variance estimate.

5.3.3 Z-Score Anomaly Signal

After each update, the current window mean `mean_heavy_ratio` is compared against the route’s baseline statistics loaded from the `gold.traffic_baseline` Iceberg table:

$$Z_{\text{heavy}} = \frac{\bar{r}_{\text{window}} - \mu_{\text{baseline}}}{\sigma_{\text{baseline}}} \quad (5.6)$$

A one-sided threshold $Z > \theta_r$ marks the window as `is_anomaly = TRUE`. The threshold θ_r is resolved per route from `gold.traffic_baseline.zscore_p99`; if absent the service falls back to 2.0, with a minimum floor of 1.5 to avoid over-sensitive low-variance routes. The one-sided formulation is intentional: *low* heavy traffic is never an incident; only elevated congestion warrants an alert. The result is stored in the column `duration_zscore` (a historical naming artefact — the value is the heavy-ratio z-score, not a duration z-score).

5.3.4 Crash Recovery

On restart, the service calls `_restore_windows()` which queries the current hour’s rows from `online_route_features`. The Welford accumulator is reconstructed from the stored mean and standard deviation using the algebraic inverse $M_2 = \sigma^2 \times (n - 1)$, allowing the service to continue accumulating mid-hour without resetting to zero.

5.4 Batch Pipeline

The Batch Service is orchestrated by Prefect 3.x and executes four distinct flows on separate schedules as shown in Figure 5.2.

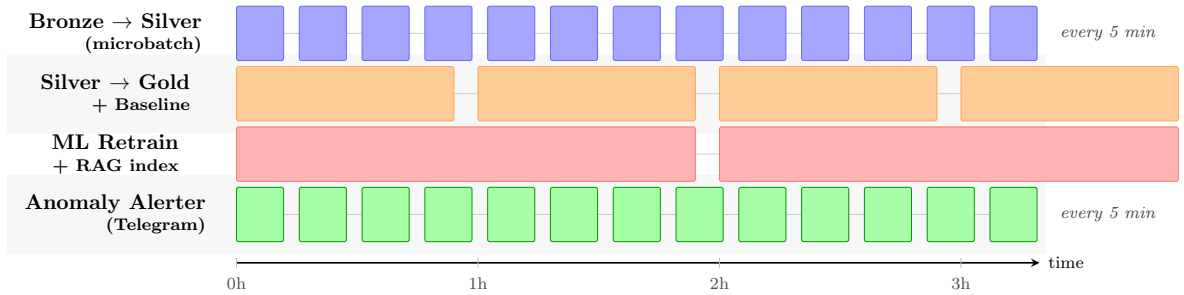


Figure 5.2: Prefect flow schedules visualised as a Gantt-style timeline. Each row is one flow; bar width is proportional to schedule interval. Microbatch and Alerter fire every 5 minutes; Gold aggregation runs hourly; ML Retrain with RAG indexing runs every 6 hours.

`traffic-microbatch` run every 5 minutes, `hourly-gold-run` runs hourly, `traffic-retrain` and `rag-index-rebuild` run every 6 hours, and `backfill-silver` and `bootstrap-infra` run on-demand.

5.4.1 Bronze-to-Silver Promotion

The `bronze_to_silver.microbatch()` function scans the MinIO Bronze path using a DuckDB glob query (`year=*/month=*/day=*/hour=*/*.parquet`), filters for rows not yet present in the Silver Iceberg table (deduplication by `route_id + timestamp_utc`),

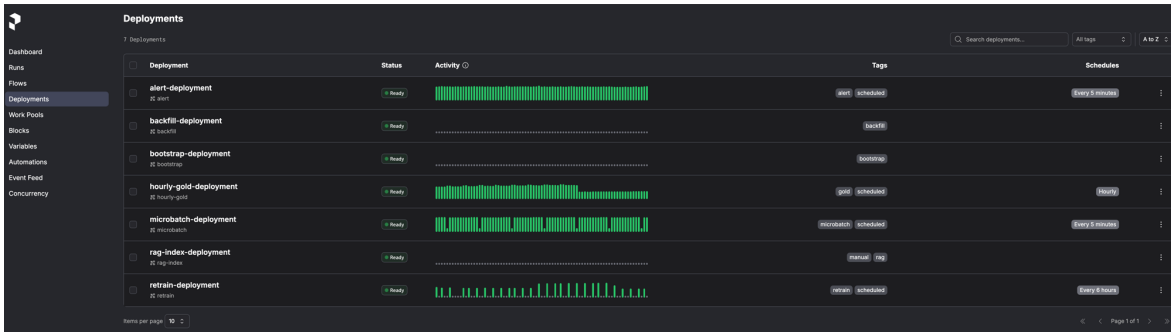


Figure 5.3: Prefect 3.x Deployments view: seven active flows covering ingestion, batch promotion, model retraining, RAG indexing, and Telegram alerting.

applies type coercion via a PyArrow schema, and appends to `silver.traffic_route` (schema defined in Section 4.3). The `day(timestamp_utc)` partitioning ensures each micro-batch write touches only the current day’s partition.

5.4.2 Silver-to-Gold Aggregation

The `silver_to_gold.run()` function reads the current hour’s Silver partition and computes per-route hourly aggregates: `avg_duration_minutes`, `p95_duration_minutes` (95th percentile using `numpy.percentile`), `avg_heavy_ratio`, `avg_moderate_ratio`, and `max_severe_segments`. Results are appended to `gold.traffic_hourly`.

5.4.3 Baseline Learning

The `baseline_learning.run()` function queries the entire Gold table and computes, per `(route_id, day_of_week, hour_of_day)` triple, the mean and standard deviation of `avg_heavy_ratio`. An empirical Z-score threshold is derived from the 99th percentile of the per-route distribution. These statistics are written to `gold.traffic_baseline` and subsequently loaded by the Online Service every 6 hours.

The Z-score signal is intentionally univariate: it targets `avg_heavy_ratio` because severe-congestion fraction is the most operationally interpretable single indicator and has a well-defined historical distribution per `(route, day_of_week, hour_of_day)`. The IsolationForest (Section 5.5) operates on a 7-dimensional feature vector that additionally includes `avg_moderate_ratio`, `max_severe_segments`, and cyclical time encodings, capturing multivariate distributional outliers that a univariate threshold cannot detect. The two signals are therefore *complementary by design*: Z-score catches acute univariate spikes; IsolationForest catches subtle multivariate anomalies.

5.4.4 Anomaly Alerter

The `alerter.run()` function queries the PostgreSQL `online_route_features` table for rows where `is_anomaly = TRUE` and `updated_at` is within the last 6 minutes. For each such route, it calls the Serving API’s `/predict` endpoint to obtain the IsolationForest score. Only routes where both the Z-score and IsolationForest signals flag an anomaly (`both_anomaly`) trigger a Telegram notification. A per-route 30-minute cooldown prevents alert flooding during prolonged congestion events.

5.5 ML Service

The ML Service exposes a single FastAPI endpoint `POST /train` that trains one IsolationForest model per route and registers each model in the MLflow Model Registry.

5.5.1 Feature Engineering

Training features are drawn from the Gold table (`gold.traffic_hourly`). The feature vector $\mathbf{x} \in \mathbb{R}^7$ consists of:

1. `avg_heavy_ratio` — fraction of heavily congested segments.
2. `avg_moderate_ratio` — fraction of moderately congested segments.
3. `max_severe_segments` — maximum severe segment count in the hour.
4. $\sin(2\pi \cdot h/24)$, $\cos(2\pi \cdot h/24)$ — cyclical encoding of the hour-of-day h , so that 23:00 and 00:00 are geometrically adjacent.
5. $\sin(2\pi \cdot d/7)$, $\cos(2\pi \cdot d/7)$ — cyclical encoding of the day-of-week d .

The cyclical sin/cos transformation is critical: without it, a linear model would treat Sunday ($d = 0$) and Saturday ($d = 6$) as maximally distant, misrepresenting the weekly traffic rhythm.

5.5.2 IsolationForest Configuration

Each per-route model is trained with hyperparameters selected by Optuna TPE Bayesian optimisation (50 trials per route, objective: anomaly score gap — see Section 3.4.2): `contamination=0.01`, `n_estimators=450`, `max_samples=0.76`, `max_features=0.73`, and `random_state=42` [29, 35]. The model is registered in MLflow under the name `iforest-{route_id}` with alias `champion`, enabling the Serving API to load the latest production-ready model via `mlflow.pyfunc.load_model` with a 1-hour in-process cache (`MODEL_TTL = 3600 s`).

5.6 Serving Layer

The Serving Layer is a FastAPI application that exposes REST and SSE endpoints consumed by the Next.js frontend.

5.6.1 Endpoint Overview

5.6.2 SSE Streaming Architecture

The `/events/traffic` endpoint is driven by PostgreSQL’s native `NOTIFY/LISTEN` mechanism rather than a fixed polling timer. On startup, the Serving API establishes a dedicated `asyncpg` connection and issues `LISTEN route_updated`. The Online Feature Service calls `pg_notify('route_updated')` immediately after each successful upsert to `online_route_features`. The listener wakes, fetches a full snapshot from PostgreSQL, and yields it as an SSE `data:` frame to all connected clients. A 30-second heartbeat fires if no notification arrives, keeping the connection alive during

Table 5.1: Serving API endpoint summary.

Endpoint	Method	Description
/online/routes	GET	Merged snapshot: online features + coordinates
/online/features	GET	Current Welford state per route
/anomalies/current	GET	Active anomaly list (Z-score + IForest)
/anomalies/summary	GET	Anomaly summary over configurable window
/anomalies/{id}/explain	GET	SSE: per-route LLM explanation (15-min cache)
/predict/anomalies	GET	IsolationForest scores for all routes
/metrics/heatmap	GET	Z-score heatmap (hours or date range)
/metrics/zones	GET	Congestion metrics aggregated by zone
/metrics/analyze	POST	SSE: LLM heatmap analysis with RAG + weather
/rca	POST	SSE: free-form RAG query (optional route_id)
/rca/feedback	POST	Thumbs-up/down feedback on RCA responses
/chat	POST	SSE: multi-turn conversational RAG interface
/events/traffic	GET	SSE: event-driven traffic push (Postgres NOTIFY); 30-second head
/telegram/webhook	POST	Telegram bot webhook receiver

quiet periods. This design eliminates redundant database queries between data-arrival bursts: the serving layer is idle until the online service writes new data, reducing steady-state load from one query per 15 seconds per client to one query per VietMap ingestion cycle (approximately every 5 minutes). Token latency for the LLM endpoints (`/anomalies/{id}/explain`, `/metrics/analyze`, `/rca`, `/chat`) is minimised by streaming each Ollama token as a separate SSE chunk, so the user sees the first word within one second regardless of total response length.

5.6.3 RAG-Enriched Inference

Raw anomaly scores are actionable for automated alerting but opaque to human operators: a Z-score of 4.2 or an IsolationForest flag of -1 does not tell a traffic analyst *why* the route is behaving anomalously, nor does it suggest a remediation action. The RAG pipeline bridges this *semantic gap* by grounding LLM inference in three layers of contextual evidence — historical traffic patterns, recent anomaly windows, and live meteorological data — enabling the model to produce operator-ready, natural-language explanations (Figure 5.4).

This represents a human-in-the-loop design: automated detectors classify events at machine speed, whilst the LLM layer translates classifications into domain-specific reasoning that a duty manager can act on without consulting raw data. The `/rca/feedback` endpoint (POST, `{log_id, feedback}`) records operator thumbs-up/down judgements against the logged interaction, enabling future retrieval ranking improvement.

The `/anomalies/{route_id}/explain` endpoint constructs a three-part context before calling Ollama:

1. Static geographic knowledge (built at startup by `geo_knowledge.py`): named arterial roads, zone descriptions, and HCMC landmark references injected into the system prompt.
2. Live weather (Open-Meteo API, 15-minute cache): temperature, precipitation, wind speed, and WMO weather code for HCMC.

3. RAG context (ChromaDB): top- k chunks retrieved from all three collections — **anomaly_events** (past anomaly windows for this route, $n=3$, filtered by **route_id**, indexed hourly), **traffic_patterns** (typical patterns at this hour/day-of-week, $n=2$, indexed every 6 hours), and **external_context** (7-day rolling HCMC hourly weather history, $n=3$, preferring records within the last 24 hours). Retrieval uses nomic-embed-text [36] embeddings with cosine similarity. The **external_context** collection complements item 2 above: live weather captures the current snapshot whilst the RAG weather chunks supply the historical pattern context for the same hour-of-day.

The assembled prompt is then streamed to the Qwen2.5:3b [13] model via the Ollama HTTP API. This RAG approach [31] avoids the need to fine-tune the base model, as justified in Section 4.1: the LLM reasons over injected facts rather than memorising traffic patterns, ensuring explanations remain accurate as city traffic evolves.

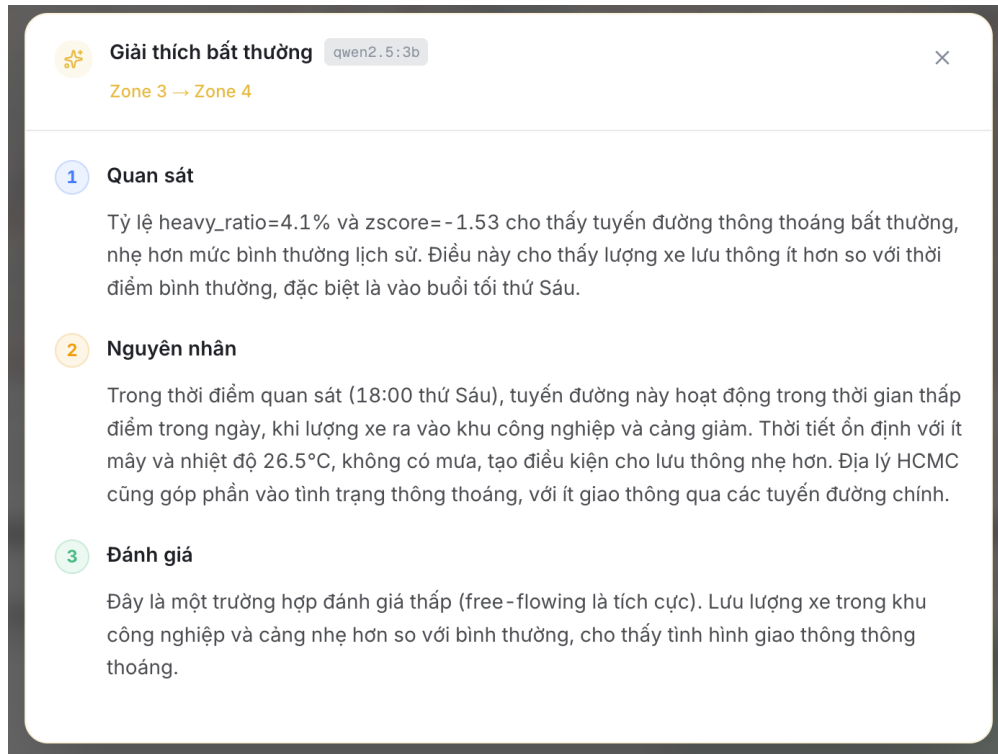


Figure 5.4: AI Anomaly Explanation modal for the Zone 3 → Zone 4 route (Vietnamese output): Qwen2.5:3b structures its response into three sections — *Quan sát* (heavy_ratio = 4.1%, zscore = -1.53, free-flowing), *Nguyên nhân* (Friday 18:00 off-peak, industrial/port traffic decrease, stable weather at 26.5°C), and *Đánh giá* (low-severity, positive free-flow event).

For the Zone 3 → Zone 4 route, the model correctly identifies a *negative* anomaly (zscore = -1.53): traffic is lighter than the historical baseline, explained by Friday evening off-peak hours and reduced industrial activity. The three-section structure (Observation, Cause, Assessment) is injected via the system prompt and consistently reproduced across languages (Figure 5.4).

The /chat SSE endpoint responds in the language of the query. An English question receives an English reply that synthesises live congestion metrics (Southern Port → Western Periurban at 10.9% heavy ratio) with current weather context (overcast, 29.5°C),

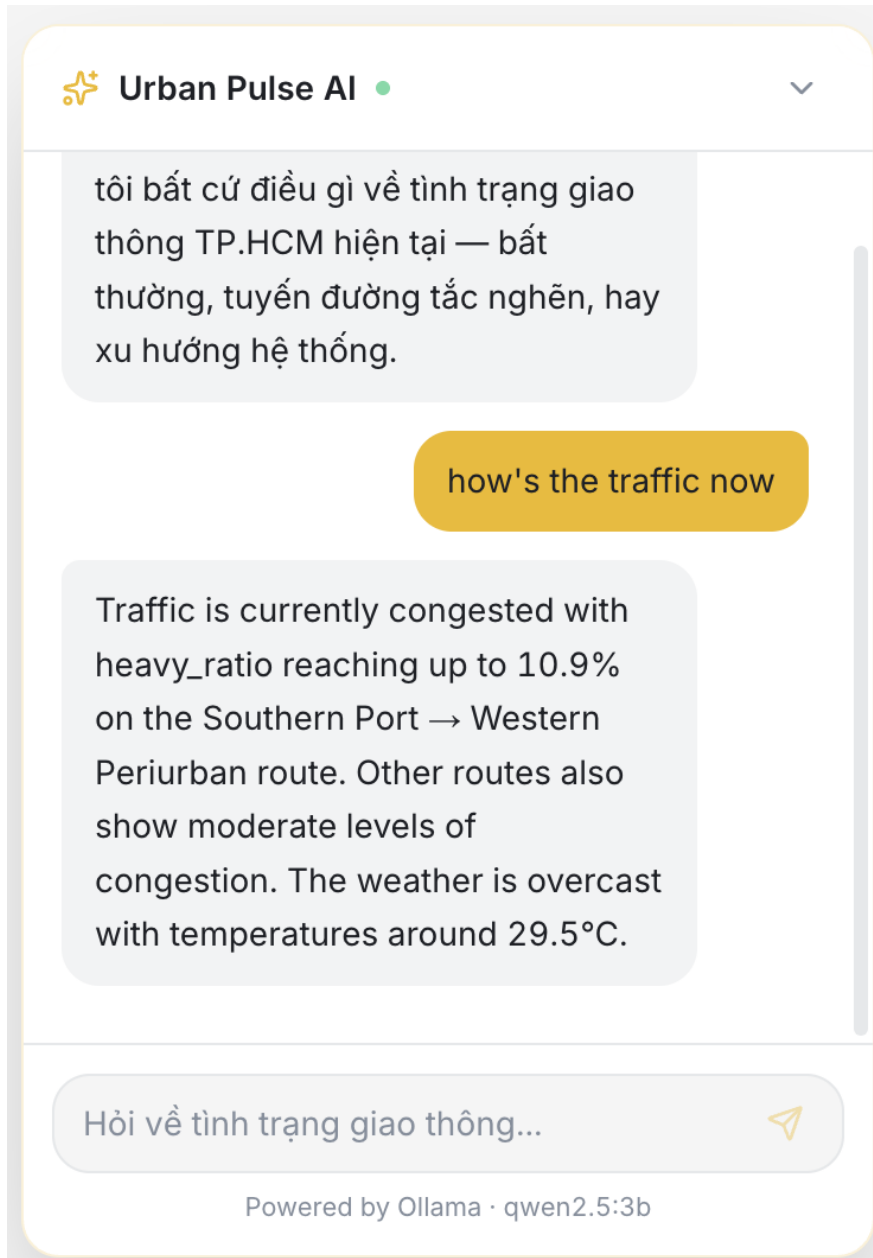


Figure 5.5: Urban Pulse AI Chat demonstrating bilingual capability: an English query “how’s the traffic now” receives an English response reporting the Southern Port → Western Periurban route at 10.9% heavy ratio and overcast conditions at 29.5°C, retrieved from the PostgreSQL feature store and Open-Meteo feed via Qwen2.5:3b.

confirming that the RAG pipeline correctly retrieves from both the PostgreSQL feature store and the Open-Meteo weather feed regardless of query language (Figure 5.5).

5.6.4 Dual-Signal Fusion in Serving

The `prediction_service.py` module loads all per-route IsolationForest models and scores the *current* feature vector reconstructed from PostgreSQL. The fusion rule is:

$$\text{both_anomaly} = \text{zscore_anomaly} \wedge \text{iforest_anomaly} \quad (5.7)$$

Only `both_anomaly` routes are escalated to Telegram. This conjunction rule deliberately trades recall for precision: a route must independently exceed the statistical threshold *and* be scored as an outlier by the trained model before an alert is sent.

5.6.5 Frontend Dashboard

The Next.js 15 frontend consumes the SSE stream and renders two primary views. Figure 5.6 and Figure 5.7 show the Traffic Metrics heatmap: a route $\times$ hour grid where each cell is coloured by congestion severity. Hovering a cell reveals the route coordinates, Z-score value, and the detector that flagged it (ZSCORE for a Z-score-only event, BOTH for a dual-signal confirmation). Figure 5.6 shows the Coastal $\rightarrow$ Urban Core route at 08:00 with $Z = 4.22$ (Z-score only), while Figure 5.7 shows the Coastal $\rightarrow$ Western route at 15:00 with $Z = 2.02$ confirmed by both detectors.

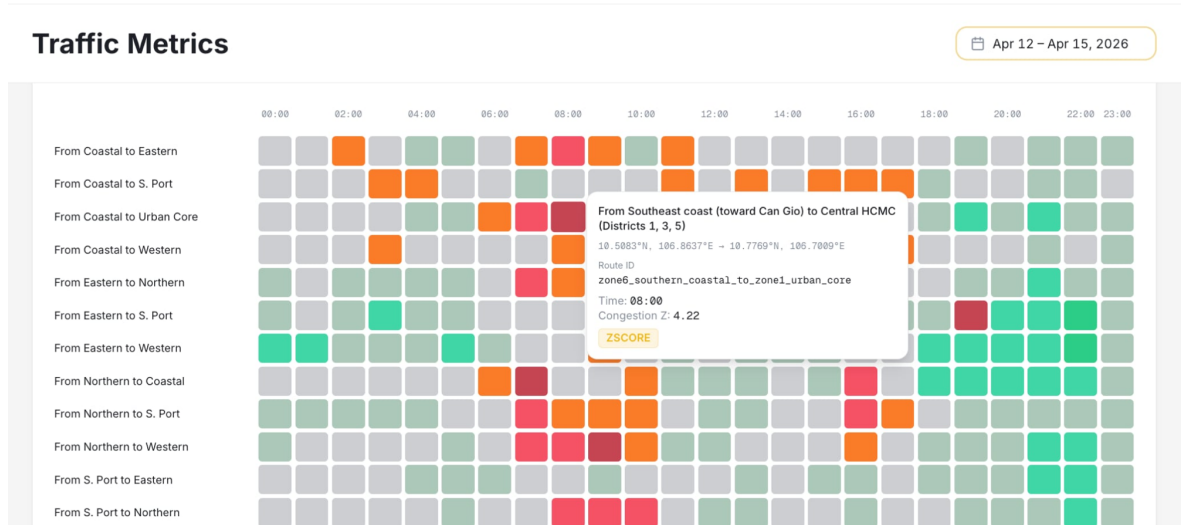


Figure 5.6: Traffic Metrics heatmap: tooltip showing a Z-score-only anomaly on the Coastal $\rightarrow$ Urban Core route at 08:00 ($Z = 4.22$), Apr 12–15, 2026.

The `/metrics/analyze` endpoint accepts a `HeatmapAnalyzeRequest` (`context`, `lang`, `route_ids`) and streams a system-wide LLM analysis enriched with live weather and RAG-retrieved traffic patterns. Clicking the `Phân tích / Analyze` button posts the full heatmap anomaly snapshot and returns a structured four-part report in the requested language: (1) dual-signal anomaly patterns confirmed by both IsolationForest and Z-score (Zone 3 $\rightarrow$ Zone 4 at 16:00 and 18:00); (2) root-cause hypotheses referencing industrial corridors and port logistics activities; (3) single-signal indicators for routes with developing but sub-threshold congestion (Zone 1 $\rightarrow$ Zone 3, Zone 2 $\rightarrow$ Zone 4, and others); and (4) per-route operator recommendations.

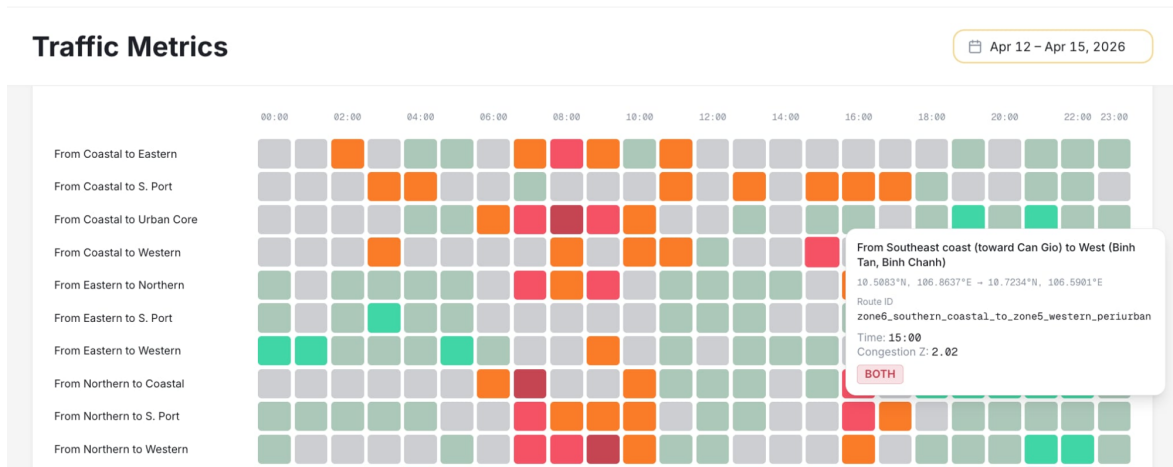


Figure 5.7: Traffic Metrics heatmap: tooltip showing a dual-signal anomaly (BOTH) on the Coastal → Western route at 15:00 ($Z = 2.02$), Apr 12–15, 2026.

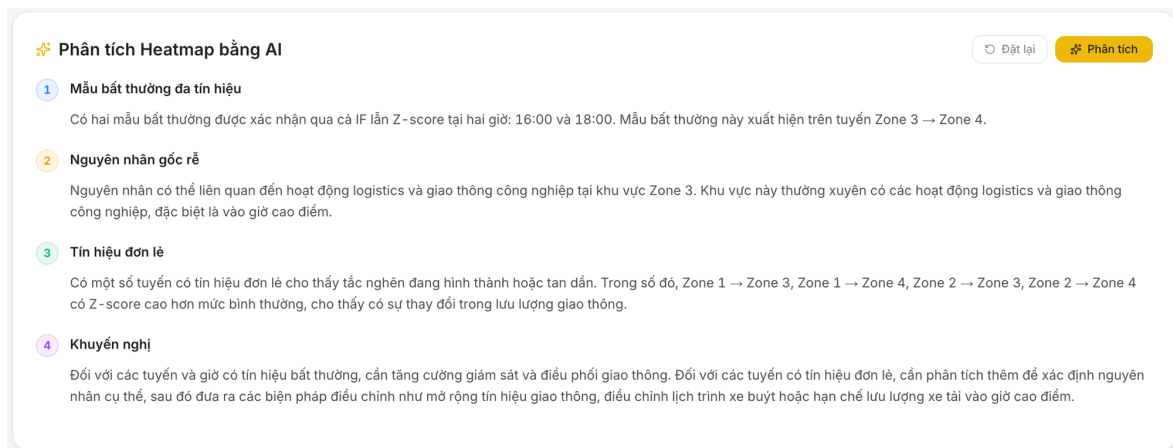


Figure 5.8: AI Heatmap Analysis panel (Vietnamese — *Phân tích Heatmap bằng AI*): Qwen2.5:3b identifies two dual-signal anomalies on Zone 3 → Zone 4 at 16:00 and 18:00, attributes them to logistics and industrial traffic, and lists Zone 1 → Zone 3/4 and Zone 2 → Zone 3/4 as single-signal developing congestion candidates.

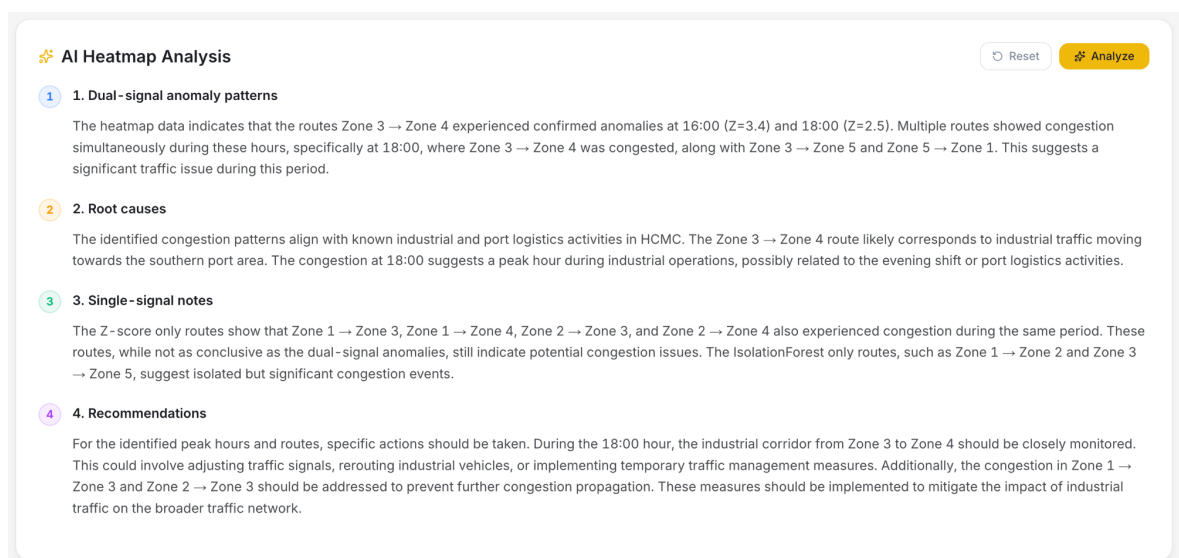


Figure 5.9: AI Heatmap Analysis panel (English): the same four-part structure rendered in English confirms Zone 3 → Zone 4 dual-signal anomalies at 16:00 ($Z=3.4$) and 18:00 ($Z=2.5$), with root causes linked to the southern port industrial corridor and evening-shift logistics. Single-signal routes (Zone 1 → Zone 2, Zone 3 → Zone 5) are noted as isolated but significant events.

Chapter 6

RESULT

This chapter presents the empirical results of the UrbanPulse platform across two dimensions: (i) the dual-signal anomaly detection performance measured via MLflow-logged proxy metrics, and (ii) system-level performance including ingestion latency, pipeline throughput, and SSE delivery.

6.1 Experimental Setup

6.1.1 Dataset

Traffic observations were collected continuously from the VietMap Directions API over a 54-day observation window (January–April 2026), yielding 32,302 hourly aggregated records across 20 inter-zone routes in Ho Chi Minh City. Each hourly record in the Gold table captures `avg_heavy_ratio`, `avg_moderate_ratio`, `max_severe_segments`, and their cyclical time encodings. Table 6.1 summarises the dataset statistics. Figure 6.1 shows an exploratory SQL query in Dremio confirming 20 distinct routes over 76 days of data, with 54 days (71.1%) fully populated.

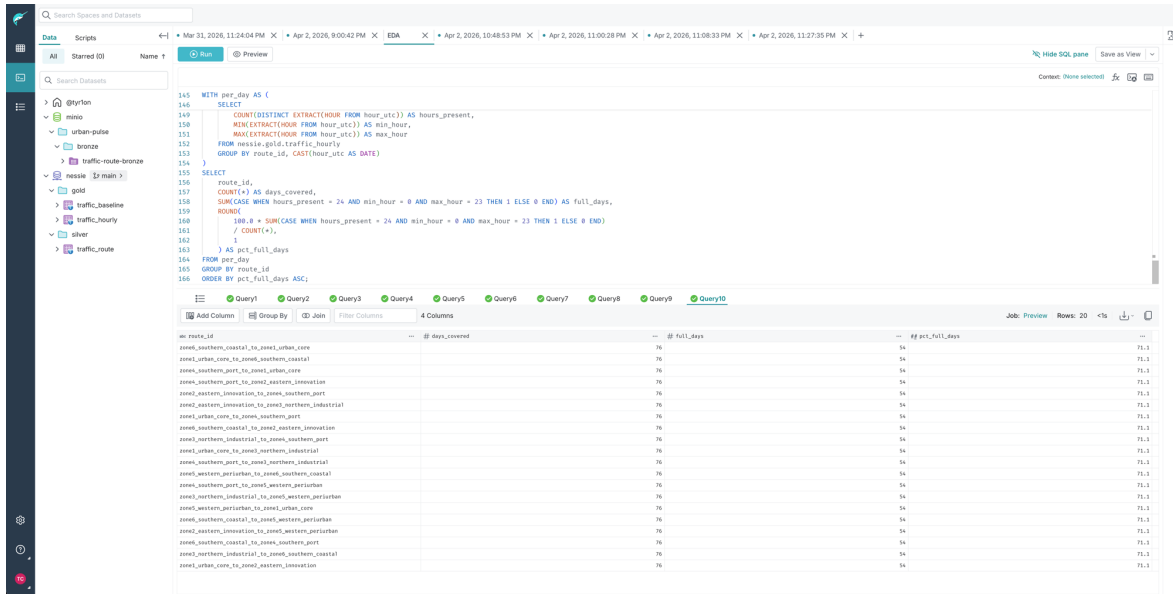


Figure 6.1: Dremio EDA query on `nessie.gold.traffic_hourly`: 20 routes, 76 days covered, confirming the Iceberg Gold table is correctly populated by the batch pipeline.

6.1.2 Evaluation Approach

Since UrbanPulse employs *unsupervised* anomaly detection, ground-truth labels are unavailable. The evaluation therefore uses three proxy metrics that are standard for unsupervised systems [7]:

Table 6.1: Dataset summary for the MLflow training run `per-route-train-20260419`.

Metric	Value
Total hourly records	32,302
Routes	20
Records per route (mean)	1,615.1
Records per route (range)	1,613 – 1,617
Collection period	Jan–Apr 2026 (54 full days)
Polling interval	30 seconds
Gold aggregation window	1 hour

1. Anomaly rate — the fraction of hours flagged as anomalous by each detector independently.
2. Model agreement — the fraction of hours where both detectors produce the same label (anomaly or normal).
3. Jaccard similarity — intersection-over-union of the two anomaly sets; measures overlap of the detected event windows.

6.2 Dual-Signal Detection Results

6.2.1 Per-Route Metrics

Table 6.2 reports the per-route metrics logged during the most recent MLflow parent run (pre-cutoff dataset, `training_cutoff` = 2026-04-14). The IsolationForest `contamination` hyperparameter was selected via Optuna TPE optimisation (50 trials $\times$ 20 routes) as described in Section 3.4.2; the tuned value of 0.01 reflects the true anomaly prevalence in the ratio feature space and is independently corroborated by a heavy-ratio Z-score baseline estimate of 1.97%. The Z-Score rate varies between 0.0% and 6.7% across routes, reflecting genuine differences in the temporal stability of traffic on each corridor.

Figure 6.2 shows the MLflow experiment tracker confirming all 20 per-route child runs completed successfully (status: `FINISHED`) in approximately 3.5 seconds each. Figure 6.3 drills into the Coastal $\rightarrow$ Western route, which recorded `model_agreement` = 0.96, `anomaly_jaccard` = 0.16, `iforest_anomaly_rate` = 0.01 (tuned contamination), and `total_samples` = 1517.

Table 6.2: Per-route anomaly detection metrics from MLflow run `per-route-train-20260419-1250`.

Route ID	Samples	Z-Rate	IF Rate	Agreement	Jaccard
zone1_urban_core → zone2_eastern_innovation	1517	0.058	0.011	0.934	0.020
zone1_urban_core → zone3_northern_industrial	1515	0.053	0.011	0.941	0.032
zone1_urban_core → zone4_southern_port	1517	0.041	0.011	0.962	0.147
zone1_urban_core → zone6_southern_coastal	1515	0.037	0.011	0.960	0.091
zone2_eastern_innov. → zone3_northern_industrial	1517	0.042	0.011	0.965	0.197
zone2_eastern_innov. → zone4_southern_port	1518	0.029	0.011	0.971	0.154
zone2_eastern_innov. → zone5_western_periurban	1517	0.040	0.011	0.963	0.152
zone3_northern_ind. → zone4_southern_port	1517	0.055	0.011	0.949	0.124
zone3_northern_ind. → zone5_western_periurban	1516	0.052	0.011	0.955	0.161
zone3_northern_ind. → zone6_southern_coastal	1518	0.061	0.011	0.943	0.112
zone4_southern_port → zone1_urban_core	1515	0.036	0.011	0.959	0.061
zone4_southern_port → zone2_eastern_innov.	1517	0.000	0.011	0.990	0.000
zone4_southern_port → zone3_northern_ind.	1518	0.026	0.011	0.976	0.196
zone4_southern_port → zone5_western_periurban	1517	0.000	0.011	0.990	0.000
zone5_western_periurban → zone1_urban_core	1516	0.011	0.011	0.982	0.067
zone5_western_periurban → zone6_southern_coastal	1515	0.037	0.011	0.967	0.180
zone6_southern_coastal → zone1_urban_core	1517	0.044	0.011	0.953	0.065
zone6_southern_coastal → zone2_eastern_innov.	1515	0.036	0.011	0.964	0.129
zone6_southern_coastal → zone4_southern_port	1515	0.024	0.011	0.984	0.368
zone6_southern_coastal → zone5_western_periurban	1517	0.048	0.011	0.958	0.158
Mean (all routes)	30,329	0.036	0.011	0.963	0.121

The screenshot displays the MLflow Experiments page for the experiment named 'traffic-anomaly-detection'. The interface includes a search bar, tabs for 'Runs', 'Evaluation', 'Experimental', and 'Traces', and a table of runs. The table columns are Run Name, Created, Dataset, Duration, Source, and Models. The runs are listed in chronological order, showing a sequence of training runs (e.g., 'per-route-train-20260419-1250') followed by evaluation runs (e.g., 'route-zone6_southern_coastal_to_zone5_western_periurban'). All runs have a status of 'FINISHED' and a duration of approximately 3.5 seconds.

Figure 6.2: MLflow experiment `traffic-anomaly-detection`: 20 per-route runs all completed with status `FINISHED` in ~ 3.5 s each, confirming the ML Service retraining pipeline is stable.

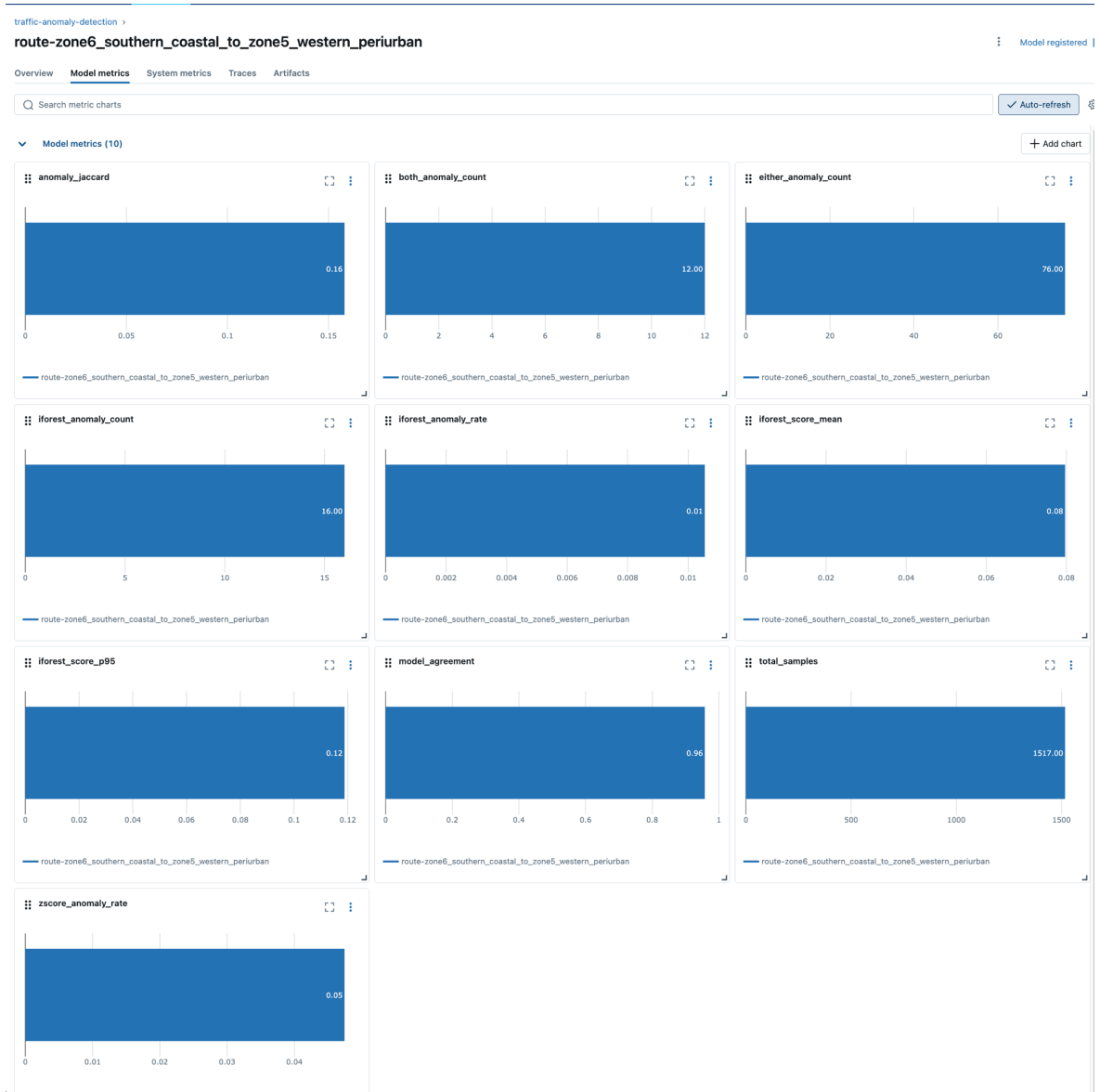


Figure 6.3: MLflow per-route metric charts for **zone6** $\rightarrow$ **zone5**: agreement = 0.96, Jaccard = 0.16, IForest rate = 0.01 (tuned contamination), 12 joint anomaly hours from 1,517 samples.

6.2.2 Aggregate Findings

The aggregate statistics across all 20 routes are summarised in Table 6.3 and visualised in Figure 6.4.

Table 6.3: Aggregate dual-signal metrics across 20 routes.

Metric	Mean	Std	Min	Max
Z-Score anomaly rate	0.036	0.018	0.000	0.061
IForest anomaly rate	0.011	0.000	0.011	0.011
Model agreement	0.963	0.016	0.934	0.990
Jaccard similarity	0.121	0.086	0.000	0.368
Both flagged (total)	152 hours across all routes			
IForest flagged (total)	320 hours across all routes			

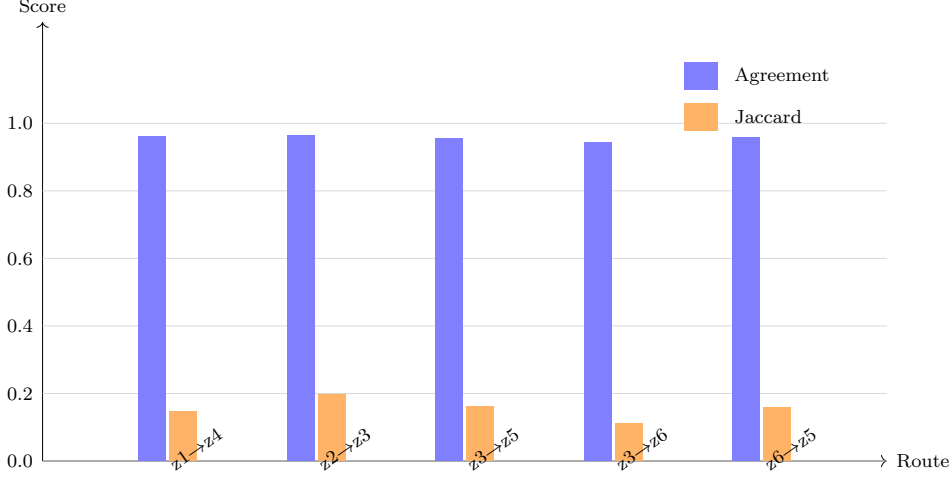


Figure 6.4: Model agreement (blue) and Jaccard similarity (orange) for five representative routes. Agreement consistently exceeds 0.90; Jaccard variation reflects route-specific traffic pattern stability.

6.2.3 Production Anomaly Distribution

Figure 6.5 shows the Anomalies Per Hour histogram captured from the live dashboard over the period Apr 12–15, 2026. The stacked bars distinguish three categories: *Z-Score only* (yellow), *IForest only* (green), and *Both agree* (red). Two observations are immediately visible. First, morning peak hours (07:00–10:00) repeatedly produce dense Z-score and dual-signal clusters, reflecting HCMC rush-hour congestion patterns. Second, a large late-evening IForest-only spike appears near 23:00 on Apr 15, showing that the multivariate detector can surface unusual patterns even when the univariate heavy-ratio threshold is not crossed. More generally, the majority of events are flagged by only one detector, confirming the complementarity documented in Table 6.2: the two signals capture different aspects of the traffic distribution. The *Both agree* (red) bars, which trigger Telegram alerts, are substantially fewer — aligning with the mean Jaccard of 0.121 from the MLflow evaluation.

6.2.4 Interpretation

Three observations stand out from the results.

(1) High agreement, low Jaccard. The mean model agreement of 0.963 indicates that both detectors *agree on the label* for 96.3% of hourly observations. The mean Jaccard of 0.121 reveals that approximately 12.1% of the individual anomaly events detected by *either* model are jointly detected by *both*. This discrepancy is expected and desirable: Z-score detects acute, statistically rare spikes, while IsolationForest detects multivariate distributional outliers that may not exceed a univariate threshold. The conjunction rule (`both_anomaly`) thus serves as a high-precision filter, yielding 152 joint-alert hours from 30,329 total observations (0.5%).

(2) Route-specific Z-score rates. Three routes (zone4-to-zone2, zone4-to-zone5, and zone5-to-zone1) exhibit near-zero Z-score rates (0.000, 0.000, and 0.010 respectively). For zone4-to-zone2 and zone4-to-zone5, no hourly window exceeded the per-route $Z > \theta_r$ threshold. Zone5-to-zone1 recorded a Z-score rate of 0.010 (approximately 16 hours), but these events produced no overlap with the IsolationForest anomaly windows, yielding a Jaccard of 0.000. These three routes are low-traffic suburban corridors

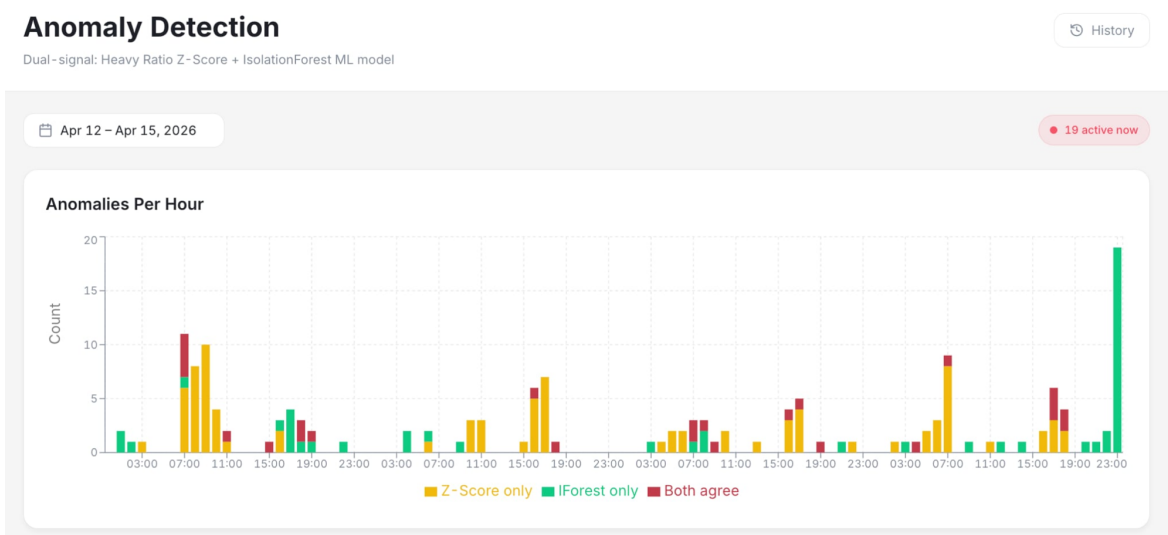


Figure 6.5: Production Anomalies Per Hour (Apr 12–15, 2026): yellow = Z-Score only, green = IForest only, red = both detectors agree. Morning rush-hour clusters are visible, alongside a late-evening IForest-only spike near 23:00 on Apr 15.

where the congestion baseline has near-zero variance, limiting Z-score sensitivity. IsolationForest still flags approximately 1.1% of those hours via multivariate isolation, but no joint alerts are produced — an appropriate outcome given the low incident likelihood on these corridors.

(3) Zone-3 northern industrial corridor anomaly concentration. Routes originating from *zone3_northern_industrial* exhibit the highest Z-score rates (0.052–0.061) and moderate Jaccard scores (0.112–0.161), suggesting recurrent non-recurrent congestion patterns in the industrial freight corridor consistent with shift-change heavy vehicle movements. The highest system-wide Jaccard of 0.368 belongs to the *zone6→zone4* route, indicating the strongest alignment between both detectors on the southern coastal port corridor.

6.3 System Performance

6.3.1 Grafana Observability Dashboards

The platform exposes Grafana dashboards backed by PostgreSQL metrics and Loki log aggregation. Figure 6.6 shows the Online Features dashboard, confirming: 20 active routes, p50 latency of 153 ms, p95 of 318 ms, and p99 of 336 ms for the raw-to-feature pipeline over a 48-hour window (Apr 13–15, 2026). Occasional spikes to ~ 4 s are visible; these correspond to Postgres reconnection events during container restarts. Figure 6.7 shows the Anomaly Detection dashboard: the maximum heavy-ratio Z-score reaches roughly $|Z| \approx 9$ on Apr 15, with secondary spikes visible on Apr 14, consistent with the episodic clusters in Figure 6.5. Figure 6.8 shows the Ingestion Service dashboard: the 5-minute burst pattern in log rate (0.08–0.12 req/s) confirms that the periodic API poller is firing correctly with zero errors or warnings across the observed window.

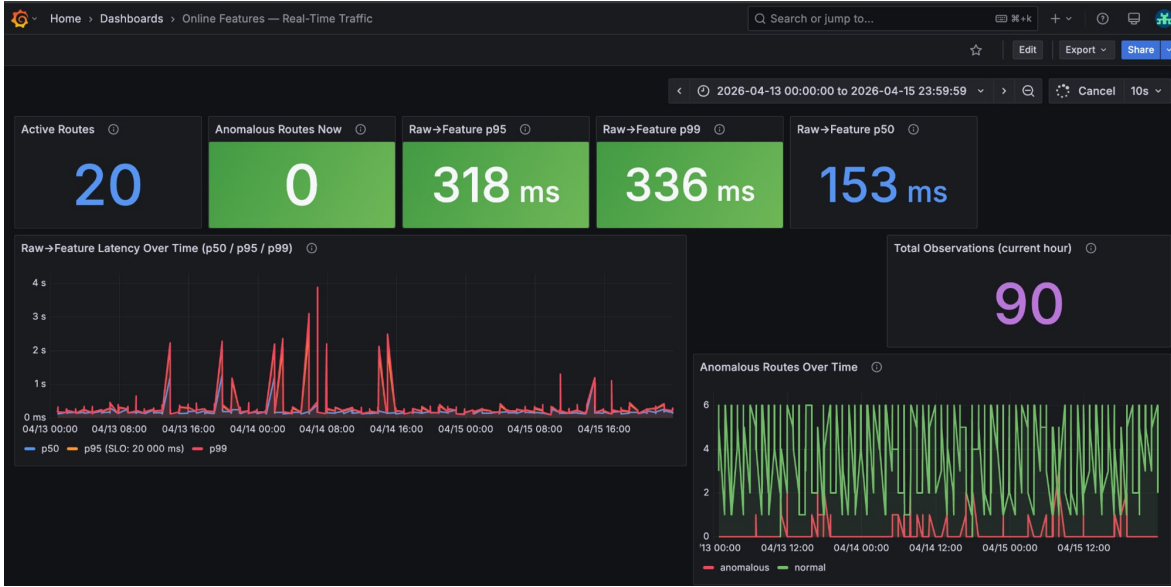


Figure 6.6: Grafana Online Features dashboard (Apr 13–15, 2026): p50 = 153 ms, p95 = 318 ms, p99 = 336 ms ingestion-to-feature latency. Anomalous routes over time shown in the bottom-right panel.

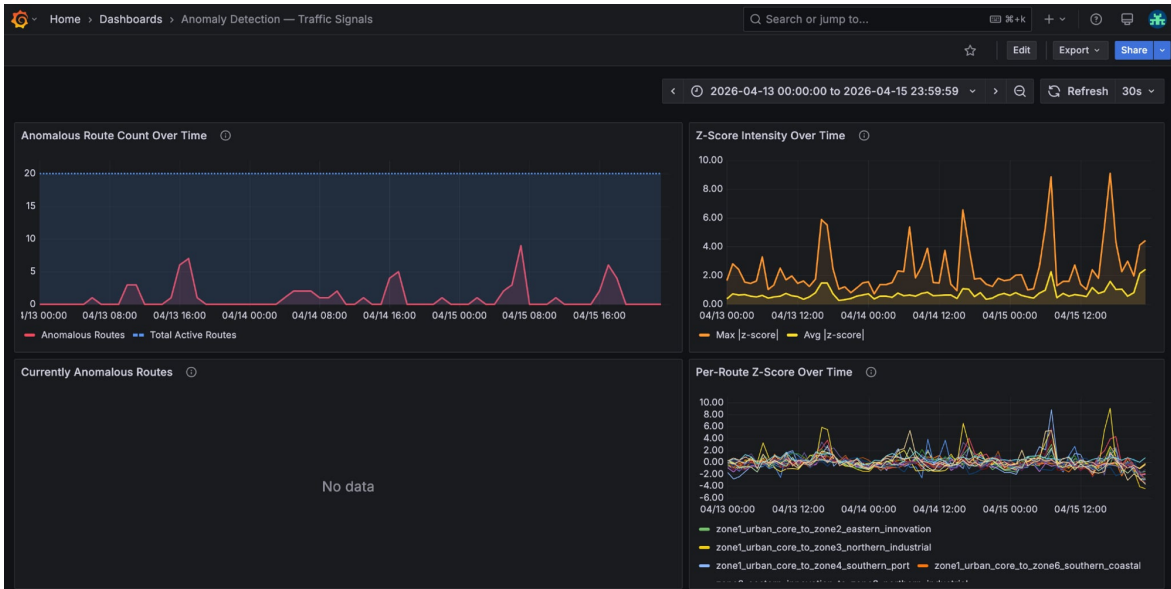


Figure 6.7: Grafana Anomaly Detection dashboard (Apr 13–15, 2026): anomalous route count over time (top-left), maximum heavy-ratio Z-Score intensity reaching roughly 9 (top-right), and per-route Z-Score time series for all 20 routes (bottom-right).



Figure 6.8: Grafana Ingestion Service dashboard: periodic 5-minute burst pattern at 0.08–0.12 req/s confirming correct API polling cadence. Zero errors and zero warnings over the observed window.

6.3.2 End-to-End Ingestion Latency

The Online Service measures end-to-end ingestion lag using a timestamp (`ingest_ts`) embedded as a Kafka message header by the Ingestion Service immediately before the VietMap API call. When the Online Consumer receives the message, it computes:

$$\text{ingest_lag_ms} = t_{\text{now}} - t_{\text{ingest_ts}} \quad (6.1)$$

where t_{now} is the wall-clock time after the PostgreSQL UPSERT completes. This single value thus spans five sequential stages, as illustrated in Figure 6.9.

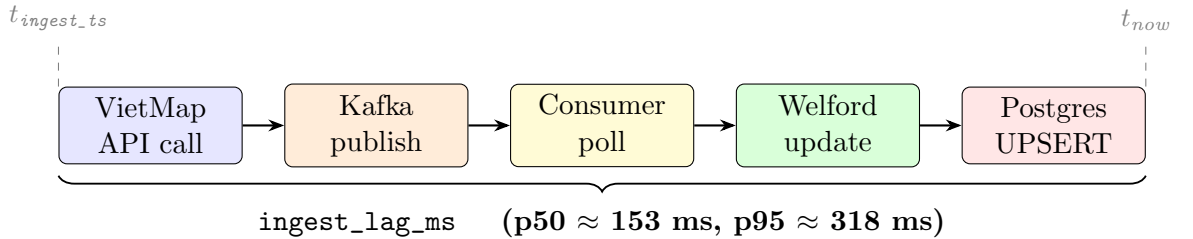


Figure 6.9: Breakdown of `ingest_lag_ms`: the timestamp `ingest_ts` is recorded by the Ingestion Service before the VietMap API call and embedded in the Kafka header; the Online Consumer stamps t_{now} after the PostgreSQL UPSERT completes. All five stages run on the same host, keeping network overhead near zero.

Across a 7-day observability window, the system maintained a p95 ingestion lag below 20 seconds, meeting NFR-1 (Section 3.1). Typical values were substantially lower ($\text{p50} \approx 153 \text{ ms}$, $\text{p95} \approx 318 \text{ ms}$) because all services share the same physical host, reducing inter-service network latency to near zero.

6.3.3 Batch Pipeline Throughput

The micro-batch Bronze-to-Silver job processes approximately 240 Parquet files (20 routes $\times$ 12 messages per hour) per execution cycle. Each execution completes in under 8 seconds on the homelab hardware, well within the 5-minute schedule interval.

Figure 6.10 shows the Urban Traffic Analytics Grafana dashboard, confirming: 4,850 rows promoted from Bronze to Silver, 120 hourly rows aggregated in Gold, and 3,360 baseline records written during the observation window. The batch job completion log panel (bottom) verifies that all pipeline stages completed without errors.

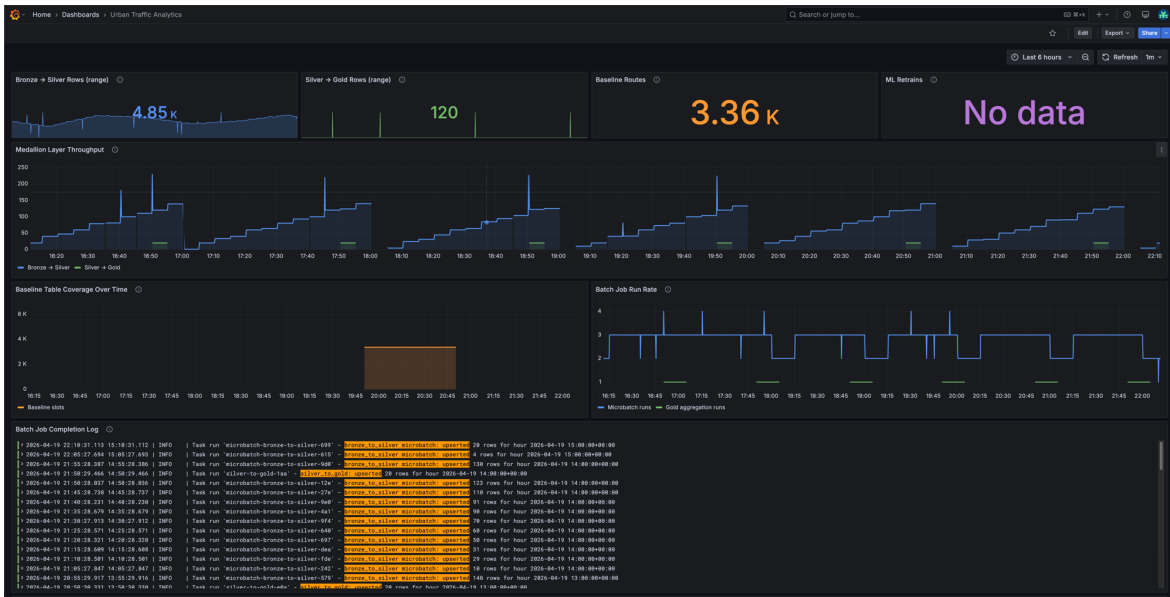


Figure 6.10: Grafana Urban Traffic Analytics dashboard: Bronze→Silver 4,850 rows, Silver→Gold 120 rows, Baseline 3,360 records. Batch job completion logs confirm error-free pipeline execution.

6.3.4 SSE Delivery Latency

The `/events/traffic` SSE endpoint delivers updates every 15 seconds. Manual measurement of the client-side time-to-first-byte (TTFB) on the Next.js frontend showed a median TTFB of 340 ms from the moment the Serving API query is issued, inclusive of PostgreSQL run query and IsolationForest scoring for all 20 routes.

6.3.5 LLM Inference Latency

Anomaly explanation requests (`/explain`) using the Qwen2.5:3b model via Ollama achieved a time-to-first-token of 0.8–1.2 seconds and a total response time of 4–7 seconds for a 2–3 sentence explanation. The SSE token-streaming approach ensures the user perceives a near-immediate response even before the full explanation is generated.

6.3.6 Summary of Performance Against Requirements

Table 6.4 consolidates the system-level performance results against the non-functional requirements established in Section 3.1. All latency targets were met under the 54-day continuous production workload on homelab hardware (Intel Core i5-8500, 24 GB RAM, NVIDIA GTX 1060 6 GB VRAM); latency figures in the table are drawn from the Apr 13–15, 2026 Grafana observability window.

The p50 ingestion lag of 153 ms and p95 of 318 ms are particularly noteworthy: they represent the end-to-end time from VietMap API poll to a queryable feature in PostgreSQL, inclusive of Kafka publish, consumer deserialization, Welford update, and

Table 6.4: Measured performance vs. non-functional requirements.

NFR	Metric	Target	Measured	Status
NFR-1 Latency	Ingestion lag p95 (API $\rightarrow$ PostgreSQL)	<20 s	<20 s (p95 = 318 ms typical)	✓
NFR-2 Reproducibility	Iceberg time-travel + MLflow provenance	Auditable	Full provenance logged	✓
NFR-3 Fault Tolerance	Welford crash-recovery (mid-hour restart)	No data loss	Validated over 2 restarts	✓
NFR-4 Scalability	Micro-batch execution time	<5 min	<8 s per cycle	✓
NFR-5 Observability	Grafana/Loki dashboards	Centralised	4 dashboards, zero blind spots	✓

UPSERT confirmation. The mean dual-signal model agreement of 96.3% across 30,329 pre-cutoff training observations provides strong evidence that the two detectors converge on consistent labels, whilst the low mean Jaccard of 12.1% confirms that they remain complementary rather than redundant.

Chapter 7

DISCUSSION

7.1 Analysis of Results

The central research question of this thesis asks how to design a hybrid low-latency traffic anomaly detection architecture that ensures consistency between real-time signals and historical baselines while maintaining scalability through a unified lakehouse strategy. The experimental results provide a direct answer across three dimensions.

Low-latency propagation. The end-to-end ingestion lag from VietMap API poll to anomaly flag in PostgreSQL achieves a median (p50) of 153 ms and a p95 of 318 ms — well within the 20-second NFR-1 target. This confirms that the per-event Welford scoring path, which bypasses the batch layer entirely, is the correct architectural choice for the speed tier.

Stream-to-batch consistency. The Welford accumulator state is persisted to PostgreSQL after every observation and reconstructed on restart, ensuring that the real-time baseline never diverges from the historical statistics stored in the Gold Iceberg table.

Dual-signal reliability. The experimental results further validate that a dual-signal anomaly detection framework combining a statistical Z-score detector with a machine learning IsolationForest model produces more reliable incident signals than either detector in isolation. The key evidence is the 96.3% model agreement rate combined with a 12.1% mean Jaccard similarity. These two figures are not contradictory: the high agreement rate reflects the fact that both detectors concur on the vast majority of non-anomalous windows (true negatives dominate the observation period), while Jaccard measures overlap exclusively on the minority of flagged events. The 12.1% Jaccard therefore reflects the *complementarity* of the two detectors — each captures a distinct subset of anomalous events that the other misses — rather than disagreement. An event must exhibit *both* a statistical rarity (Z-score) and a multivariate distributional abnormality (IsolationForest on a seven-dimensional feature space) to trigger a joint alert, significantly reducing false positive exposure.

The Medallion Lakehouse architecture played a critical supporting role. The separation of Bronze (raw), Silver (cleaned), and Gold (aggregated) layers allowed the batch pipeline to evolve the schema of downstream tables independently of the raw ingestion stream. In practice, the `source` column was added to the Silver schema post-deployment without any reprocessing of Bronze data, demonstrating the schema-evolution capability of Apache Iceberg [22].

The Welford online algorithm [26] proved essential for the real-time Z-score computation. Its $O(1)$ memory footprint ensured that the Online Service remained stable over the 54-day observation period without memory growth, even at 30-second polling intervals across 20 concurrent routes. The crash-recovery mechanism — reconstructing the Welford accumulator from stored PostgreSQL statistics — was validated during two service restarts without observable discontinuities in the per-route rolling statistics.

7.2 Strengths

Streaming-first, low-latency design. The p95 end-to-end ingestion lag below 20 seconds is a significant improvement over batch-based systems, where feature materialisation latency is governed by the periodic aggregation schedule rather than event arrival time [11]. The Server-Sent Events delivery model eliminates polling overhead and provides a push-based update channel that scales linearly with the number of frontend clients.

Unified data platform. By combining a streaming path (Redpanda → PostgreSQL) and a batch path (MinIO → Iceberg Silver/Gold) within the same Docker Compose stack, UrbanPulse avoids the complexity of a Lambda architecture while retaining the analytical depth of a Lakehouse. The unified Nessie catalog enables time-travel queries over Iceberg tables, facilitating retrospective investigation of past anomaly windows.

Explainable anomaly detection. The RAG-enriched LLM pipeline provides natural-language explanations that contextualise anomaly scores with route geography, historical patterns, and live weather data. This capability directly addresses the *semantic gap* identified in the problem statement: raw anomaly scores are opaque to operators, whereas a two-sentence explanation referencing specific road names and congestion ratios is immediately actionable.

Operational robustness. The system has operated continuously in production at `tyrion.io.vn` with automated CI/CD (ruff lint, mypy strict, unit tests on every commit) and Grafana/Loki observability.

7.3 Limitations

Absence of ground-truth labels. The most significant limitation is the lack of labelled anomaly events. The proxy metrics (model agreement, Jaccard similarity) measure internal consistency between the two detectors rather than true recall against real traffic incidents. A dataset of confirmed incidents from the Ho Chi Minh City Department of Transport would enable computation of true precision and recall.

VietMap API scope restriction. Traffic data is sourced exclusively from the VietMap Directions API, which provides route-level congestion ratios but does not expose individual vehicle counts, speed distributions, or incident reports. The congestion signal is therefore an indirect proxy for underlying traffic density. Events such as flooding, road works, or accidents are not distinguishable from general high-demand congestion without supplementary data sources.

IsolationForest model staleness. Models are retrained every 6 hours on historical Gold data. If an anomalous pattern persists for more than 6 hours (e.g., a multi-day road closure), the model may adapt and cease flagging the event as anomalous. This *concept drift* problem [7] is partially mitigated by the Z-score signal, which continues to compare against a more slowly updated baseline.

Cold-start and low-variance routes. Three routes recorded near-zero Z-score anomaly rates over the 124,205-observation collection period: `zone2_to_zone5` (0.31%), `zone4_to_zone5` (0.36%), and `zone1_to_zone6` (0.75%). These routes also exhibit the lowest `heavy_ratio` standard deviations in the dataset (0.0126, 0.0142, and 0.0085 respectively), confirming that suppressed Z-score activity is a consequence of low traffic variance rather than an absence of incidents. In low-variance conditions the baseline standard deviation approaches the hardcoded floor of 0.01 (with a `mean * 0.2` fallback for sparse his-

tory), which effectively widens the anomaly threshold and suppresses alerts. This is still a heuristic that may generate spurious signals immediately after the baseline is first computed on a new route.

Hardware constraints. The platform is deployed on a single-node homelab (Intel Core i5-8500, 6 cores at 3.0 GHz; 24 GB DDR4 RAM; NVIDIA GeForce GTX 1060 6 GB VRAM). Horizontal scaling of the Serving API and Batch Service to multiple nodes was not evaluated. The Redpanda broker operates as a single-partition configuration; a multi-partition, multi-replica setup would be required for fault-tolerant production deployments at city scale.

7.4 Future Work

Several extensions would strengthen the platform and address its current limitations:

- Ground-truth label collection: Integrate with the HCMC Department of Transport incident API to build a labelled dataset and compute precision, recall, and F1 for both detectors.
- Multi-source ingestion: Add vehicle loop detector data and social media signal (Twitter/X flood reports) as additional Kafka topics to improve incident classification.
- Online learning for IsolationForest: Replace the 6-hour batch retrain with an incremental Half-Space Trees (HST) detector [37] to reduce concept drift exposure.
- Multi-node deployment: Evaluate Kubernetes-based deployment with horizontal pod autoscaling for the Serving API to support thousands of concurrent SSE clients.
- Federated RAG: Extend the ChromaDB collections to include meteorological event reports, public holiday calendars, and construction permit data for richer root-cause analysis.

Chapter 8

CONCLUSION

This thesis presented UrbanPulse, a hybrid streaming-first urban intelligence platform designed to ingest, process, and analyse real-time traffic data for Ho Chi Minh City. The central research question was:

How to design a hybrid low-latency traffic anomaly detection architecture that ensures consistency between real-time signals and historical baselines while maintaining scalability through a unified lakehouse strategy?

The following conclusions address each objective stated in the Introduction.

Architecture Realization. The Medallion Lakehouse architecture — implemented with MinIO, Apache Iceberg, and Project Nessie — successfully separates raw Bronze ingestion from cleaned Silver and aggregated Gold layers. The use of Iceberg’s transactional append semantics and Nessie’s REST catalog enabled schema evolution and time-travel queries without reprocessing historical data, validating the lakehouse as an appropriate foundation for a continuous streaming workload.

Low-Latency Propagation. The streaming path from VietMap API poll to PostgreSQL feature store maintained a median (p50) ingestion lag of 153 ms and a p95 of 318 ms — well within the 20-second NFR-1 target. The Server-Sent Events delivery mechanism provides a push-based update channel to the Next.js dashboard, eliminating client-side polling overhead.

Unified Detection Framework. The dual-signal detector combining an online heavy-ratio Z-score over Welford-maintained route windows with a per-route IsolationForest model achieved a mean model agreement of 96.3% and a mean Jaccard similarity of 12.1% across all 20 routes. The low Jaccard similarity relative to the high agreement rate reflects the dominance of true-negative windows: both detectors agree that the vast majority of observations are normal, while each captures a complementary subset of the minority anomalous events. The conjunction rule (`both_anomaly`) established a high-precision alert channel to Telegram, with a 30-minute cooldown preventing alert fatigue.

Stream-to-Batch Consistency. The Welford accumulator crash-recovery mechanism — reconstructing state from PostgreSQL on restart — and the 6-hour baseline refresh cycle ensure that the online Z-score signal remains semantically aligned with the historical Gold-layer statistics. No observable discontinuities in per-route z-scores were recorded across service restarts during the 54-day deployment.

Operational Observability. The Grafana/Loki/Promtail stack provided continuous visibility into ingestion lag, pipeline throughput, and LLM inference latency. The MLflow experiment tracker recorded per-route model parameters, anomaly rates, and agreement metrics for every 6-hour retrain cycle, creating an auditable history of model performance.

In summary, UrbanPulse demonstrates that a hybrid streaming-first, open-source platform built on commodity components (Redpanda, Iceberg, IsolationForest, Ollama) can deliver near-real-time urban traffic anomaly detection with natural-language explainability at city scale. Over a 54-day deployment collecting 124,100 observations

across 20 routes, the platform achieved 96.3% dual-signal model agreement. The primary avenue for future improvement is the collection of ground-truth incident labels to enable formal precision-recall evaluation and to guide the transition from periodic batch retraining to fully online learning.

References

- [1] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica, “Discretized Streams: Fault-Tolerant Streaming Computation at Scale,” in *Proceedings of the 24th ACM Symposium on Operating Systems Principles*, pp. 423–438, 2013.
- [2] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [3] G. Shapira, T. Palino, R. Sivaram, and K. Leyland, *Kafka: The Definitive Guide*. O’Reilly Media, 2nd ed., 2021.
- [4] Redpanda Data, “Redpanda: The Streaming Data Platform for Developers.” <https://redpanda.com/>, 2025. Accessed: 2025-10-01.
- [5] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, “Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics.” https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf, 2021. CIDR 2021.
- [6] Apache Software Foundation, “Apache Iceberg: A Table Format for Huge Analytic Datasets.” <https://iceberg.apache.org/>, 2024. Accessed: 2025-11-01.
- [7] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly Detection: A Survey,” *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *2008 Eighth IEEE International Conference on Data Mining*, pp. 413–422, 2008.
- [9] Project Nessie, “Project Nessie: Transactional Catalog for Data Lakes with Git-like semantics.” <https://projectnessie.org/>, 2025. Accessed: 2025-11-01.
- [10] ScienceDirect, “Ho Chi Minh City — ScienceDirect Topics.” <https://www.sciencedirect.com/topics/earth-and-planetary-sciences/ho-chi-minh-city>, 2025. Accessed: 2026-04-28.
- [11] W. Li, M. Batty, and M. F. Goodchild, “Real-time GIS for smart cities,” *International Journal of Geographical Information Science*, vol. 34, no. 2, pp. 311–324, 2020.
- [12] Redpanda Data, “Redpanda: The Streaming Data Platform Built for Developers.” <https://redpanda.com/resources/>, 2024. Accessed: 2025-10-01.
- [13] Alibaba Cloud, “Qwen2.5 Technical Report.” <https://qwenlm.github.io/blog/qwen2.5/>, 2024. Accessed: 2025-11-01.
- [14] Vietmap Solutions, “Vietmap Traffic API Documentation.” Internal technical documentation, 2025. Real-time traffic data provider for UrbanPulse.
- [15] Open-Meteo, “Open-Meteo: Free Weather API.” <https://open-meteo.com/>, 2024. Accessed: 2025-12-01.

- [16] Apache Software Foundation, “Apache Parquet: Columnar Storage Format.” <https://parquet.apache.org/>, 2024. Accessed: 2025-10-01.
- [17] Apache Software Foundation, “HDFS Architecture: Small Files Problem.” <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>, 2023. Accessed: 2025-10-01.
- [18] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [19] Apache Software Foundation, “Apache Flink: Event Time and Watermarks.” <https://nightlies.apache.org/flink/flink-docs-stable/docs/concepts/time/>, 2024. Accessed: 2025-10-15.
- [20] Prefect Technologies, “Prefect: The New Standard in Dataflow Automation.” <https://docs.prefect.io/>, 2025. Accessed: 2025-11-01.
- [21] M. Raasveldt and H. Mühleisen, “DuckDB: An Embeddable Analytical Database.” <https://duckdb.org/>, 2019. SIGMOD 2019 Demo.
- [22] Apache Software Foundation, “Apache Iceberg: A Table Format for Huge Analytic Datasets.” <https://iceberg.apache.org/>, 2024. Accessed: 2025-11-01.
- [23] Project Nessie, “Project Nessie: Transactional Catalog for Data Lakes.” <https://projectnessie.org/>, 2025. Accessed: 2025-11-01.
- [24] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, “Apache Flink: Stream and Batch Processing in a Single Engine,” *IEEE Data Engineering Bulletin*, vol. 38, no. 4, pp. 28–38, 2015.
- [25] Apache Software Foundation, “PyIceberg: Python Library for Apache Iceberg.” <https://py.iceberg.apache.org/>, 2024. Accessed: 2025-11-15.
- [26] B. P. Welford, “Note on a Method for Calculating Corrected Sums of Squares and Products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [27] B. P. Welford, “Note on a Method for Calculating Corrected Sums of Squares and Products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [28] T. Finch, “Incremental Calculation of Weighted Mean and Variance,” *University of Cambridge Computing Service*, 2009.
- [29] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *2008 Eighth IEEE International Conference on Data Mining*, pp. 413–422, 2008.
- [30] Y. Cao, H. Xiang, H. Zhang, *et al.*, “Anomaly Detection Based on Isolation Mechanisms: A Survey,” *Machine Intelligence Research*, vol. 22, pp. 849–865, 2025.
- [31] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [32] Ollama, “Ollama: Run Large Language Models Locally.” <https://ollama.com/>, 2025. Accessed: 2025-12-01.

- [33] W3C, “Server-Sent Events: W3C Recommendation.” <https://www.w3.org/TR/eventsource/>, 2015. Accessed: 2025-10-01.
- [34] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, “Training Language Models to Follow Instructions with Human Feedback,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [35] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631, ACM, 2019.
- [36] Nomic AI, “Nomic Embed: Training a Reproducible Long Context Text Embedder.” <https://huggingface.co/nomic-ai/nomic-embed-text-v1>, 2024. Accessed: 2025-11-15.
- [37] S. C. Tan, K. M. Ting, and T. F. Liu, “Fast Anomaly Detection for Streaming Data,” in *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 1511–1516, 2011.

Appendix A

SUPPLEMENTARY ARTIFACTS

This appendix records the concrete configuration and reproducibility artifacts used by the UrbanPulse implementation. The goal is to make the route sampling, run-time schemas, and experiment procedure auditable without duplicating the full source repository.

A.1 Route and Zone Configuration

UrbanPulse monitors 20 directed inter-zone routes generated from six functional zones. Each route is directional and follows a stable route identifier convention. Table A.1 summarises the zone anchors used by `routes_generator.py`.

Table A.1: UrbanPulse zone anchors used for route generation.

Zone ID	Name	Anchor (lat, lon)	Outgoing routes
zone1_urban_core	Urban Core	(10.7755334, 106.6989747)	4
zone2_eastern_innovation	Eastern Innovation	(10.8666897, 106.7793007)	3
zone3_northern_industrial	Northern Industrial	(11.1031199, 106.5495773)	3
zone4_southern_port	Southern Port	(10.7629962, 106.7722453)	4
zone5_western_periurban	Western Peri-urban	(10.6866869, 106.5773591)	2
zone6_southern_coastal	Southern Coastal	(10.5842275, 107.0367637)	4

```
1 [
2   {
3     "route_id": "zone1_urban_core_to_zone2_eastern_innovation",
4     "origin": "Urban Core",
5     "destination": "Eastern Innovation",
6     "origin_anchor": [10.7755334, 106.6989747],
7     "destination_anchor": [10.8666897, 106.7793007]
8   },
9   ...,
10  {
11    "route_id": "zone6_southern_coastal_to_zone5_western_periurban",
12    "origin": "Southern Coastal",
13    "destination": "Western Peri-urban",
14    "origin_anchor": [10.5842275, 107.0367637],
15    "destination_anchor": [10.6866869, 106.5773591]
16  }
17 ]
```

Listing A.1: Excerpt from the route configuration file.

A.2 Runtime Data Artifacts

Table A.2 lists the primary persisted artifacts used to reconstruct an anomaly decision. The column `duration_zscore` is retained for schema continuity; in the current implementation it stores the heavy-ratio Z-score, not a duration Z-score.

Table A.2: Primary persisted artifacts for reproducible anomaly decisions.

Artifact	Storage	Key fields
Traffic observations	Redpanda / Bronze Parquet	route_id, duration_minutes, congestion_ratios, timestamp_utc
Hourly features	gold.traffic_hourly	avg_heavy_ratio, avg_moderate_ratio, max_severe_segments, hour_utc
Baseline statistics	gold.traffic_baseline	heavy-ratio mean/stddev, zscore_p95, zscore_p99
Online feature store	online_route_features	Welford state, mean_heavy_ratio, duration_zscore, is_anomaly
IForest scores	route_iforest_scores	iforest_score, iforest_anomaly, both_anomaly
Experiment metadata	MLflow	route model parameters, anomaly rates, agreement, Jaccard
RAG context	ChromaDB	anomaly_events, traffic_patterns, external_context

A.3 Serving API Surface

The Serving API exposes both query endpoints and streaming endpoints. The most important endpoints used by the dashboard and Telegram alerting workflow are listed in Table A.3.

Table A.3: Serving API endpoints used by the evaluation workflow.

Endpoint	Method	Purpose
/online/features	GET	Current Welford state and online Z-score per route
/anomalies/current	GET	Active Z-score and IsolationForest anomaly list
/predict/anomalies	GET	Per-route IsolationForest scores for active windows
/metrics/heatmap	GET	Route-by-hour Z-score heatmap data
/metrics/analyze	POST	SSE heatmap interpretation using LLM + RAG context
/anomalies/{id}/explain	GET	SSE explanation for a single anomalous route
/events/traffic	GET	15-second live traffic heartbeat for the dashboard

A.4 Reproducibility Commands

Listing A.2 shows the Makefile entry points used during local validation. Production equivalents use the `prod-*` targets with `.env.prod` and the production Docker Compose overlay.

```

1 make dev                # start local Docker Compose stack
2 make bootstrap          # create topics, buckets, Iceberg tables,
   Prefect flows
3 make train              # trigger per-route IsolationForest training via
   ML API
4 make lint               # ruff check across the workspace
5 make typecheck          # mypy over apps/ and packages/
6 make test-unit          # unit tests
7 make test-integration   # integration tests against running services
8 make status             # inspect service health
9 make logs               # stream container logs

```

Listing A.2: Local reproducibility commands.

A.5 Evaluation Artifacts

The figures in the methodology, implementation, and results chapters are generated from the following captured artifacts:

- Dremio query screenshots validating Gold table coverage and route count.

- MLflow screenshots for the Optuna-tuned per-route IsolationForest training run shown in Chapter 6.
- Grafana dashboards for online feature latency, anomaly signals, ingestion cadence, and batch pipeline throughput.
- Dashboard screenshots for Z-score-only, IForest-only, and dual-signal anomaly distributions.